Bachelor's Thesis

SoftMax or Sigmoid: Impact Analysis of Output Functions in Deep Neural Networks on Accuracy and Uncertainty for the Semantic Segmentation of Earth Observation Imagery



Piel 0456819
Computer Vision & Remote Sensing

15.02.2026

Summary

Thema der Arbeit

SoftMax oder Sigmoid: Einfluss Analyse von Ausgabefunktionen bei Deep Neural Networks auf Genauigkeit und Unsicherheit für die Semantische Segmetierung von Erdbeobachtungsbildern

Stichworte

Unsicherheitsschätzung, semantische Segmentierung, convolutional neural networks, Fernerkundung, synthetic aperture radar (SAR)

Kurzzusammenfassung

Tiefe neuronale Netze sind die führende Technik zur Analyse von Erdbeobachtungsbildern. Während diese Techniken in der Praxis aufgrund ihrer Genauigkeit genutzt werden, wäre es begehrenswert, sie nicht nur für ihre Genauigkeit zu nutzen, sondern auch für die Unsicherheits Quantifizierung, die sie Menschen, die Entscheidungen treffen müssen, bieten kann. Das könnte es ihnen ermöglichen, informierte Entscheidungen in risikoreichen Situationen, wie aktiven Waldbränden oder Überflutungen, zu treffen. Jedoch wird tiefen neuronalen Netzen, die mit der SoftMax-Funktion trainiert wurden, schon lange nachgesagt, dass sie Vorhersagen mit zu viel Selbstbewusstsein treffen. Das Ziel dieser Arbeit ist es, zu testen, ob moderne tiefe neuronale Netze, die mit der SoftMax-Funktion trainiert wurden, immer noch zu selbstsichere Vorhersagen ausgeben. Zusätzlich wird in dieser Arbeit evaluiert, ob tiefe neuronale Netze, die mit der Sigmoid-Funktion trainiert wurden, genutzt werden können, um bessere Unsicherheits Quantifizierungen zu erhalten, im Vergleich zu Netzen, die mit der SoftMax-Funktion trainiert wurden. Die Ergebnisse zeigen, dass Netze, die mit den SoftMax- und Sigmoid-Funktionen trainiert wurden, in Bezug auf die Genauigkeit vergleichbare Ergebnisse ausgeben. Im Gegensatz zu vorherigen Arbeiten zeigen Netze, die mit der SoftMax-Funktion trainiert wurden, keine Tendenz, Vorhersagen mit zu viel Selbstvertrauen zu treffen, insbesondere nicht, nachdem sie kalibriert wurden. Netze, die mit der Sigmoid-Funktion trainiert wurden, zeigen eine größere Menge an Vorhersagen mit niedrigem Selbstvertrauen, im Vergleich zu Netzen, die mit der SoftMax-Funktion trainiert wurden. Insgesamt zeigen diese Netze jedoch keine erhöhte Unsicherheits Quantifizierungs Genauigkeit.

Title of Thesis

SoftMax or Sigmoid: Impact Analysis of Output Functions in Deep Neural Networks on Accuracy and Uncertainty for the Semantic Segmentation of Earth Observation Imagery

Keywords

Uncertainty estimation, semantic segmentation, convolutional neural networks, remote sensing (RS), synthetic aperture radar (SAR)

Abstract

Deep neural networks are the state-of-the-art in the analysis of Earth observation imagery. While these tools are used in practice due to their accuracy, it would be desirable to not only use them for their accuracy, but also for the uncertainty quantification they can provide human decision makers. This could enable them to make informed decisions in high-stakes scenarios, such as active wildfires or floods. However, deep neural networks trained using SoftMax have long been said to provide overconfident predictions. The objective of this work is to test whether modern deep neural networks that are trained using the SoftMax function for the purpose of semantic segmentation of Earth observation imagery still provide overconfident predictions. Additionally, this work evaluates if deep neural networks trained using the Sigmoid function can be used to provide better uncertainty quantification compared to networks trained using SoftMax. The results show that networks trained using the SoftMax and Sigmoid functions provide predictions of comparable accuracy. In contrast to previous works, the networks that were trained using SoftMax do not show a tendency to provide overconfident predictions, particularly following calibration. Networks trained using the Sigmoid function show a higher amount of low confidence predictions compared to networks trained using the SoftMax function. However, these networks do not show an overall higher uncertainty quantification accuracy.

Contents

List of Figures III

List of Tables V

Abbreviations VI

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Research Question . . . 2

2 Related Work 4
- 2.1 Uncertainty Estimation Methodologies . . . 4
  - 2.1.1 Test-time data augmentation . . . 4
  - 2.1.2 Bayesian Neural Networks . . . 4
  - 2.1.3 Ensemble Learning . . . 4
  - 2.1.4 Monte Carlo Dropout . . . 5
- 2.2 Other work . . . 5

3 Methods 7
- 3.1 Uncertainty . . . 7
  - 3.1.1 Calibration . . . 7
  - 3.1.2 Temperature Scaling . . . 8
- 3.2 Networks . . . 8
  - 3.2.1 U-Net . . . 8
  - 3.2.2 Deep Residual U-Net (Deep ResUnet) . . . 9
  - 3.2.3 Dilated Convolutional Layer . . . 10
  - 3.2.4 AtrousDenseUDeconvNet . . . 11

4 Experiments 14
- 4.1 Metrics . . . 14
  - 4.1.1 Mean Intersection over Union . . . 14
  - 4.1.2 Expected Calibration Error . . . 14
  - 4.1.3 Reliability Diagram . . . 15
- 4.2 Datasets . . . 15
  - 4.2.1 Potsdam Dataset . . . 15
  - 4.2.2 2025 Data Fusion Dataset . . . 16
  - 4.2.3 Dataset Pre-Processing . . . 16
- 4.3 Experiment 1: Accuracy and Uncertainty Quantification . . . 17
- 4.4 Experiment 2: Scalar Temperature Scaling vs Vector Temperature Scaling . . . 17

5 Results **18**
5.1 Accuracy 18
5.2 Uncertainty 21
5.3 Discussion 23

6 Conclusion **30**

References **31**

Appendix **35**

List of Figures

3.1 U-Net Architecture. **Bold** numbers indicate changes of tensor size by max pool layers and upsampling, with the numbers describing the post-execution tensor size relative to the model input tensor size. Non-bold numbers indicate the number of output channels of convolutional layers. Blue arrows depict skip connections. Green arrows depict normal connections. Based on PlotNeuralNet [32]. 10

3.2 Deep ResUnet Architecture. **Bold** numbers indicate changes of tensor size by convolutional layers and upsampling, with the numbers describing the post-execution tensor size relative to the model input tensor size. Non-bold numbers indicate the number of output channels of convolutional layers. Blue arrows depict skip connections. Green arrows depict normal connections. Based on PlotNeuralNet [32]. 11

3.3 Dilated Convolutional Layer with kernel size 3 and dilation 2. The lower blue grid shows the input image for the example of the first output pixel. The dark blue tiles are used to calculate the output value. The upper teal grid represents the layer output. Source: Dumoulin and Visin [34]. 12

3.4 AtrousDenseUDeconvNet Architecture. **Bold** numbers indicate changes of tensor size by average pool layers, max pool layers, upsampling, and transposed convolutional layers, with the numbers describing the post-execution tensor size relative to the model input tensor size. Non-bold numbers indicate the number of output channels of convolutional layers. Blue arrows depict skip connections. Green arrows depict normal connections. Based on PlotNeuralNet [32]. 13

4.1 The top diagram shows a reliability diagram. The blue and pink bars show accuracy and mean confidence of the bin, respectively. The red diagonal shows the identity, signifying ideal calibration. The bottom diagram shows the sample distribution, giving the number of samples per bin on a logarithmic scale. 15

5.1 Reliability diagram for the uncalibrated U-Net using SoftMax on the Potsdam dataset 21

5.2 Reliability diagram for the uncalibrated U-Net using Sigmoid on the Potsdam dataset 21

5.3 Reliability diagram for the scalar temperature scaled U-Net using SoftMax on the Potsdam dataset 21

5.4 Reliability diagram for the scalar temperature scaled U-Net using Sigmoid on the Potsdam dataset 21

5.5 Reliability diagram for the vector temperature scaled U-Net using SoftMax on the Potsdam dataset . . . 22
5.6 Reliability diagram for the vector temperature scaled U-Net using Sigmoid on the Potsdam dataset . . . 22
5.7 Reliability diagram for the vector temperatued scaled Deep ResUnet using SoftMax on the Data Fusion SAR dataset . . . 22
5.8 Reliability diagram for the vector temperatued scaled Deep ResUnet using Sigmoid on the Data Fusion SAR dataset . . . 22
5.9 Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using SoftMax on the Data Fusion RGB dataset . . . 23
5.10 Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using Sigmoid on the Data Fusion RGB dataset . . . 23
5.11 Reliability diagram for the uncalibrated U-Net using SoftMax on the Data Fusion RGB dataset . . . 25
5.12 Reliability diagram for the uncalibrated U-Net using Sigmoid on the Data Fusion RGB dataset . . . 25
5.13 Example of the output of models trained on the Data Fusion SAR dataset; (a) class predictions of models trained using SoftMax; (b) uncertainty of models trained using SoftMax; (c) class predictions of models trained using Sigmoid; (d) uncertainty of models trained using Sigmoid . . . 26
5.14 Example of the output of models trained on the Data Fusion RGB dataset; (a) class predictions of models trained using SoftMax; (b) uncertainty of models trained using SoftMax; (c) class predictions of models trained using Sigmoid; (d) uncertainty of models trained using Sigmoid . . . 28
5.15 Example of the output of models trained on the Potsdam dataset; (a) class predictions of models trained using SoftMax; (b) uncertainty of models trained using SoftMax; (c) class predictions of models trained using Sigmoid; (d) uncertainty of models trained using Sigmoid . . . 29

List of Tables

4.1 Dataset pre-processing details per dataset 16
5.1 Accuracy metrics of models using SoftMax on the Potsdam dataset 18
5.2 Accuracy metrics of models using Sigmoid on the Potsdam dataset 19
5.3 Accuracy metrics of models using SoftMax on the Data Fusion SAR dataset 19
5.4 Accuracy metrics of models using Sigmoid on the Data Fusion SAR dataset 19
5.5 Accuracy metrics of models using SoftMax on the Data Fusion RGB dataset 20
5.6 Accuracy metrics of models using Sigmoid on the Data Fusion RGB dataset 20
5.7 Accuracy metrics of models trained by Su, Li, Ma, *et al.* [11] on the Potsdam dataset 24
5.8 Accuracy metrics of models trained by the top 3 participants in the Data Fusion contest. 24

Abbreviations

Abbreviation	Meaning
BNN	Bayesian Neural Network
DNN	Deep Neural Network
ECE	Expected Calibration Error
EOI	Earth Observation Imagery
FCN	Fully Convolutional Network
GRSS	Geoscience and Remote Sensing Society
GSD	Ground Sampling Distance
mIoU	mean Intersection over Union
NN	Neural Network
SAR	Synthetic Aperture Radar
SCE	Static Calibration Error

1 Introduction

This work explores the accuracy and uncertainty quantification precision of Deep Neural Networks (DNNs), which use either SoftMax or Sigmoid as their output function. The models are used to predict which class each pixel in the Earth Observation Imagery (EOI) belongs to. Finally, the impact of the choice of output function on the accuracy and uncertainty prediction quality is analyzed.

1.1 Motivation

The analysis of EOI can be used for a variety of applications, from the classification of land usage in urban areas to the creation of road maps [1], [2]. In order to perform these tasks, semantic segmentation is used, that is, the assignment of a class, from a preselected list of classes, to each pixel in the EOI. For land usage analysis, a number of classes can be used, common ones being vegetation, tree, building, car, and water. For road maps, it is likely to be a binary classification, using only the classes road and other. Historically, a number of different approaches have been used for the semantic segmentation of EOI, such as State Vector Machines [3], Random Decision Forests [4], and Conditional Random Fields [5].
However, during the 2010s Fully Convolutional Networks (FCNs), DNNs using only convolutional layers, due to their large improvements in accuracy in comparison to the state of the art, started to get predominantly used in computer vision [6], [7], and soon after began to get used in the analysis of EOI [2], [8]. These networks also have the advantage that, at the same time that they output their class prediction, they also estimate the probability that the pixel shows an object belonging to a specific class. Accurate estimations by the model of how certain it is can allow users to utilize the network outputs while being aware of the risk of the network being wrong. This kind of information can be used in applications, such as flood- and burned area classification, for which these kinds of models are being developed [9], [10]. The operators of the networks need to know how confident it is, based on the EOI, that there is a real natural disaster happening in the depiction, so that they can allocate resources based on it. However, most models for non-binary classification problems are trained using SoftMax [10], [11]

$$\sigma(x)_i = \frac{\exp x_i}{\sum_{j=1}^{K} \exp x_j}, \tag{1.1}$$

where σ is the SoftMax function, exp is the standard exponential function, x is the SoftMax input, usually the model output, i is the index in SoftMax's output vector, and K is the number of classes. SoftMax has been widely used for its state of the art performance in accuracy, but multiple papers have previously found that the accuracy of the uncertainty estimation of networks trained using SoftMax is unreliable [12], [13]. This is due to how

networks are usually trained with SoftMax in classification. When the model output is compared to the one-hot encoded ground truth by the loss function, it tries to increase the output of the model for the correct class and decrease all other outputs. This leads to high model accuracy, but has the issue that the SoftMax function will almost never output 1 as a value. Because of this, the loss function will in theory repeatedly increase the model output, leading to overconfidence. However, in practice, this might be mitigated by a sufficiently large dataset, such that all values are increased when they are correct and decreased when they are not, and this way reach an equilibrium around the actual probability.
Alternative methods for uncertainty estimation have been developed, such as test-time data augmentation [14], Bayesian Neural Networks [15]–[17], and Deep Ensembles [18]. The drawback of these methods is that, while they may have better uncertainty estimation, they all require multiple forward passes for each inference, leading to noticeably higher runtimes.
A possible method that only requires a single inference pass per image while having better uncertainty estimation properties could be the Sigmoid output function:

$$\sigma(x) = \frac{\exp x}{1 + \exp x}, \tag{1.2}$$

where σ is the Sigmoid function, x is the function input, and exp is the standard exponential function. It is usually used in binary applications or for in multilabel contexts [19], because it evaluates each class separately, allowing it to assign high probabilities to multiple classes. This exact property is what might make it attractive for uncertainty estimation, especially relating to EOI. Since Sigmoid is applied to each class by itself, it can be easier for it to assign low probabilities to all classes or high probabilities to multiple classes, whereas SoftMax is said to mainly assign a high probability to a single class.

1.2 Research Question

In this work, multiple DNNs are trained on remote sensing datasets using SoftMax and Sigmoid as their output function, and then evaluated on their performance in accuracy and uncertainty quantification metrics. The first goal of this work is to test whether recent DNNs using SoftMax perform poorly on uncertainty quantification metrics in the field of remote sensing. The second goal is to investigate whether using SoftMax is the correct approach for training DNNs on EOI. These images often contain pixels that have ambiguity towards the class to which they belong, e.g., a tree that is over a road. SoftMax based models would attempt to classify such cases as a single class. However, using Sigmoid as the output function allows the model to give both the tree and the road class a high probability, empowering later analysis to recognize that either there is a tree and a road, or that the network is uncertain about its prediction. Alternatively, the model has the ability to assign

low probabilities to all classes in cases where it cannot determine the class based on the data. With this in mind, the research objective is to measure whether using the Sigmoid function can be used to achieve improved uncertainty quantification performance.

2 Related Work

2.1 Uncertainty Estimation Methodologies

2.1.1 Test-time data augmentation

Data augmentation has long been used during training to create larger datasets. This approach is used to increase the accuracy and robustness of Neural Networks (NNs) [20], [21]. The concept of producing additional images from the dataset cannot only be used during training, but also during inference. Using these augmentations, such as flipping and rotations, the network outputs multiple, different predictions, which can then be combined to obtain the model's uncertainty. Wang, Li, Aertsen, *et al.* [14] introduce a mathematical formalization of test-time data augmentation and use it to reduce overconfident, incorrect predictions, thereby achieving higher overall segmentation accuracy compared to using a single model without test-time data augmentation or a model using Monte Carlo Dropout.

2.1.2 Bayesian Neural Networks

Bayesian Neural Networks (BNNs) are an approach for uncertainty quantification, which is highly focused on uncertainty accuracy. BNNs [15]–[17], unlike other NNs, do not try to predict the class of an object; instead, they predict the distribution of parameters of the model. With each data point in the training dataset, this distribution is updated to try to more accurately estimate which sets of model parameters are most likely to predict the training data. For inference, one approach is to sample models from the distribution of parameters and use both the output of the models and the probability of the model itself to estimate the uncertainty of the prediction. Disadvantages of BNNs can be high inference time due to multiple forward passes, as well as high computation costs to train on large datasets [22].

2.1.3 Ensemble Learning

In ensemble learning, rather than training a single network for inference, multiple networks are trained, and their outputs are combined to obtain the prediction. Hansen and Salamon [18] introduced this method with the idea that the consensus of multiple networks is more likely to be correct than the prediction of a single network. To combine the predictions of the networks, there are multiple possible methods, with the most popular being averaging their outputs, or outputting the one that was predicted most frequently by the networks. For training ensembles, it is either possible to train multiple networks directly, or to train a single large network and then train multiple smaller networks using its outputs [23]. Ensemble learning has the disadvantage that it has a higher training and inference time due to the use of multiple networks.

2.1.4 Monte Carlo Dropout

Monte Carlo Dropout was introduced in Gal and Ghahramani [24] as a method to approximate the distribution a BNN would output, without the additional computational effort it takes to train one. It simulates the sampling of models a BNN does using dropout layers. Dropout layers were introduced in Srivastava, Hinton, Krizhevsky, *et al.* [25] as a method to prevent the overfitting of NNs by randomly setting the output of the neurons in the previous layer to zero at a given rate. These layers were originally designed to only be active during training and only provide some simple normalization during inference, to account for all neurons being active. However, in Monte Carlo Dropout, dropout layers are not only used during training, but also during inference, to simulate the sampling of different networks from a distribution a BNN would produce. Similar to a BNN, multiple forward passes are performed during inference to obtain the uncertainty estimate. Monte Carlo Dropout has the drawback that the quality of its uncertainty estimate is very dependent on choosing an appropriate dropout rate [26]. This can be achieved by performing a grid search to optimize the dropout rate. However, since this requires the training of multiple networks, it proves to be expensive when training large networks.

2.2 Other work

Ludwig and Hänsch [27] test multiple model types, including pure SoftMax, Test-Time Data Augmentation, and Deep Ensembles, for uncertainty quantification for Synthetic Aperture Radar (SAR) flood image classification, and analyze multiple approaches of obtaining the uncertainty from the model output. They conclude that all their tested uncertainty quantification approaches lead to similar results, contradicting earlier works [12], [13] that found that using SoftMax leads to poor uncertainty quantification results.
Haas and Rabus [28] compare Monte Carlo Dropout, Deep Ensembles, and pure SoftMax models, which are trained on a road classification SAR dataset, based on accuracy and uncertainty quantification metrics. They also analyze how the performance of the models changes if the outputs are filtered based on model confidence, with non-confident outputs being sent for human review. They find that Monte Carlo Dropout performs better on their test set for uncertainty quantification than the pure SoftMax model but attribute this difference mostly to it having better accuracy. Overall they conclude that Deep Ensembles have the best performance in their tests, and that even if the training of them is more expensive that they are likely the best option for real-world applications.
Weiss and Tonella [29] attempt to develop guidelines for the usage of uncertainty quantification of DNN. In doing so, they list multiple possible approaches for calculating uncertainty based on SoftMax logits. For a single model, they list three approaches for determining the confidence: the basic approach of using the highest SoftMax outputs, taking the difference between the two highest SoftMax outputs, and calculating the entropy over

the outputs. For multi-model approaches, they list 4 ways to calculate the confidence for classification problems: taking the mean of all SoftMax outputs, calculating the percentage of models that did not choose the predicted class as their prediction, taking the average amount of information of the predicted distributions, or calculating the mutual information metric based on the prediction and the network's posterior distribution.
Możejko, Susik, and Karczewski [30] attempt to improve the uncertainty estimation of SoftMax based on Inhibited SoftMax [31]. They add an additional parameter to the SoftMax calculation:

$$\mathrm{IS}_a(x)_i = \frac{\exp x_i}{\sum_{j=1}^{K} \exp x_j + \exp a}, \tag{2.1}$$

where $x \in \mathbb{R}^K$ is the function input, $a \in \mathbb{R}$ is the inhibition factor, i is the output index of the Inhibited SoftMax function, exp is the standard exponential function, and K is the number of classes. They define the uncertainty of the network as the inverse of the sum of probabilities assigned to the real outputs, which is equal to the assigned probability to the inhibition output node:

$$\mathrm{P}_a^u(x) = 1 - \frac{\sum_{i=1}^{K} \exp x_i}{\sum_{i=1}^{K} \exp x_i + \exp a} = \frac{\exp a}{\sum_{i=1}^{K} \exp x_i + \exp a}, \tag{2.2}$$

where P_a^u is the function determining the uncertainty of the network, x is the network output, a is the inhibition factor, exp is the standard exponential function, and K is the number of classes. They find that, for shallow Multi Layer Perceptrons, this method can provide results comparable to more computationally expensive methods like Monte Carlo Dropout, and Deep Ensembles. But they also state that this methodology is, for now, not viable for DNN as well as larger architectures.

3 Methods

3.1 Uncertainty

A NN is a complex mathematical function f_θ, where the network's weights and biases θ are the parameters. In the case of semantic segmentation, the network maps an image $x \in \mathbb{R}^{m \times n}$, out of the set of all possible images $\mathbb{X}$ to a prediction $\hat{x} \in \mathbb{R}^{m \times n \times K}$, out of the set of all possible predictions $\mathbb{Y}$, where the input image contains $m \times n$ pixel and there K classes, i.e.

$$f_\theta : \mathbb{X} \to \mathbb{Y} \qquad f_\theta(x) = \hat{x}. \tag{3.1}$$

This prediction is then processed by an output function, such as SoftMax or Sigmoid, so that it can then be interpreted as a probability assigned by the model. In this thesis, the approach used to obtain the confidence of the model from the output probabilities is to use the largest probability as the confidence, i.e.:

$$P^c = \max_i \sigma(\hat{x})_i, \tag{3.2}$$

where P^c is the prediction confidence, max is the maximum function, σ is the output function, and $\hat{x}$ is the model output. The confidence is then used to calculate the uncertainty as its inverse probability, via:

$$P^u = 1 - P^c, \tag{3.3}$$

where P^u is the prediction uncertainty, and P^c is the prediction confidence.

3.1.1 Calibration

Calibration is the term for the measurement of the quality of uncertainty quantification. Meaning that the confidence of the network P^c equals the true probability of the input belonging to the class. In the event that 100 samples are classified by the network with a confidence of 0.8, it is expected that 80 of the samples were correctly classified. Calibration is mathematically defined as:

$$\mathbb{P}(Y = \hat{Y} \mid P^c = p) = p \quad \forall p \in [0,1], \tag{3.4}$$

i.e. the probability that the actual class Y is the same as the predicted class $\hat{Y}$, given that the network predicted it with confidence p, should be p for a perfectly calibrated network [12]. In theory, miscalibration, or calibration error, can be measured by analyzing the probability distribution, via:

$$\mathbb{E}\left[\mathbb{P}(Y = \hat{Y} \mid P^c = p) - p\right], \tag{3.5}$$

in practice, this probability distribution is unknown, such that it can only be approximated.

3.1.2 Temperature Scaling

In practice, achieving perfect calibration is impossible. However, there are a number of methods to improve the calibration of networks. Temperature Scaling is a method introduced by Guo, Pleiss, Sun, *et al.* [12] for uncertainty quantification. It is a post-hoc method to change the confidence of the network to improve its calibration. It utilizes a non-negative scalar parameter T. The temperature scales the output probabilities by dividing the output logits of the network as in:

$$\hat{q}_i = \sigma(z_i/T), \tag{3.6}$$

where $\hat{q}_i$ is the predicted probability of class i, σ is the output function, and z_i is the output logit of class i. For $T = 1$, the output is identical to unscaled uncertainty quantification. For $T < 1$, if SoftMax is used, it will increase the network's confidence in its prediction. If Sigmoid is used, it will increase the confidence in high certainty predictions and lower confidence in low certainty ones. Conversely, if $T > 1$, when using SoftMax, it will lower the highest confidence and increase the others. For Sigmoid networks, it will decrease the confidence in predictions greater than 0.5 and increase it in predictions lower than 0.5.
This method is used for the calibration of networks, since it does not affect their accuracy. Additionally, to better improve the calibration of Sigmoid networks, this paper investigates the usage of vector temperature scaling, i.e.:

$$\hat{q}_i = \sigma(z_i/T_i), \tag{3.7}$$

changing the temperature from a scalar to a vector, thereby independently scaling each class. Since Sigmoid calculates the confidence of each class independently, potentially resulting in different calibration errors per class, this method could improve the calibration of Sigmoid models. However, since different classes are being scaled with different factors, this method is no longer accuracy preserving. In the experiments, it was observed that the impact of this is negligible.

3.2 Networks

For the semantic segmentation of EOI, Fully Convolutional Networks [6] are frequently used [2], [11], that is, NNs that use only convolutional layers as trainable parameters. In this work, the following three variants of Fully Convolutional Networks are used for their high accuracy and ease of use.

3.2.1 U-Net

Architecture

Ronneberger, Fischer, and Brox [7] introduced the U-Net architecture for the semantic segmentation of biomedical images. It became widely used [2], [11], [27] for its high

accuracy for a wide variety of segmentation applications. The architecture gets its name from the typical U-shape of the network structure (see Figure 3.1). The architecture can be divided into two segments: the contracting path on the left and the expanding path on the right. The contracting path extracts information from the image with convolutional networks in each level and lowers the size of the data with max pooling layers between levels. The last step in the contracting path directly feeds into the expanding path, scaling the data up using bilinear upsampling, combining it with skip connections from the expanding path, and processing it with additional convolutional layers.

Model

The model introduced by Ronneberger, Fischer, and Brox [7] used two convolutional layers in each level in the contracting path, as well as the expanding path, with kernel size 3 and stride 1. The first convolutional layer in the first level of the contracting path outputs 64 channels, with the first convolutional layer in every subsequent level of the contracting path multiplying the number of channels by two.

The model combines the upsampling with a convolutional layer to create "up-convolutions," first doubling the size of the feature maps, then halving the number of feature maps through a convolutional layer with kernel size 2 and stride 1. The network crops the output of the contracting path levels before it is stacked upon the feature maps in the expanding path if it is larger. Symmetrically to the contracting path, the first convolutional layer in each level of the expanding path halves the number of channels. The expanding path contains an additional output convolutional layer, projecting the number of channels to the number of classes of the segmentation problem.

The convolutional layers in the original U-Net model do not use padding. Because of this, it only outputs a segmentation map for the center of the image. Subsequent papers with networks based on the U-Net architecture make use of padding to avoid this [2], [11].

3.2.2 Deep Residual U-Net (Deep ResUnet)

Zhang, Liu, and Wang [2] built upon the U-Net architecture by incorporating additional skip connections. Their architecture adds a skip connection before the start of each level of convolutional layers to its end. It removes one level of the contracting and the expanding path, the max pooling layers, and the convolutional layers after upsampling, reducing the total number of convolutional layers from 23 to 15. Instead of using max pooling layers, the first convolution layer in each level of the contracting path has a stride of 2, causing the same halving of the image size. Unlike the original model, the Deep ResUnet utilizes padding in its convolutional layers, preserving the size of the data, allowing it to output segmentation maps the same size as its input. Despite the reduced number of layers, it reaches better results on their road extraction benchmark on the Massachusetts Roads dataset [33].

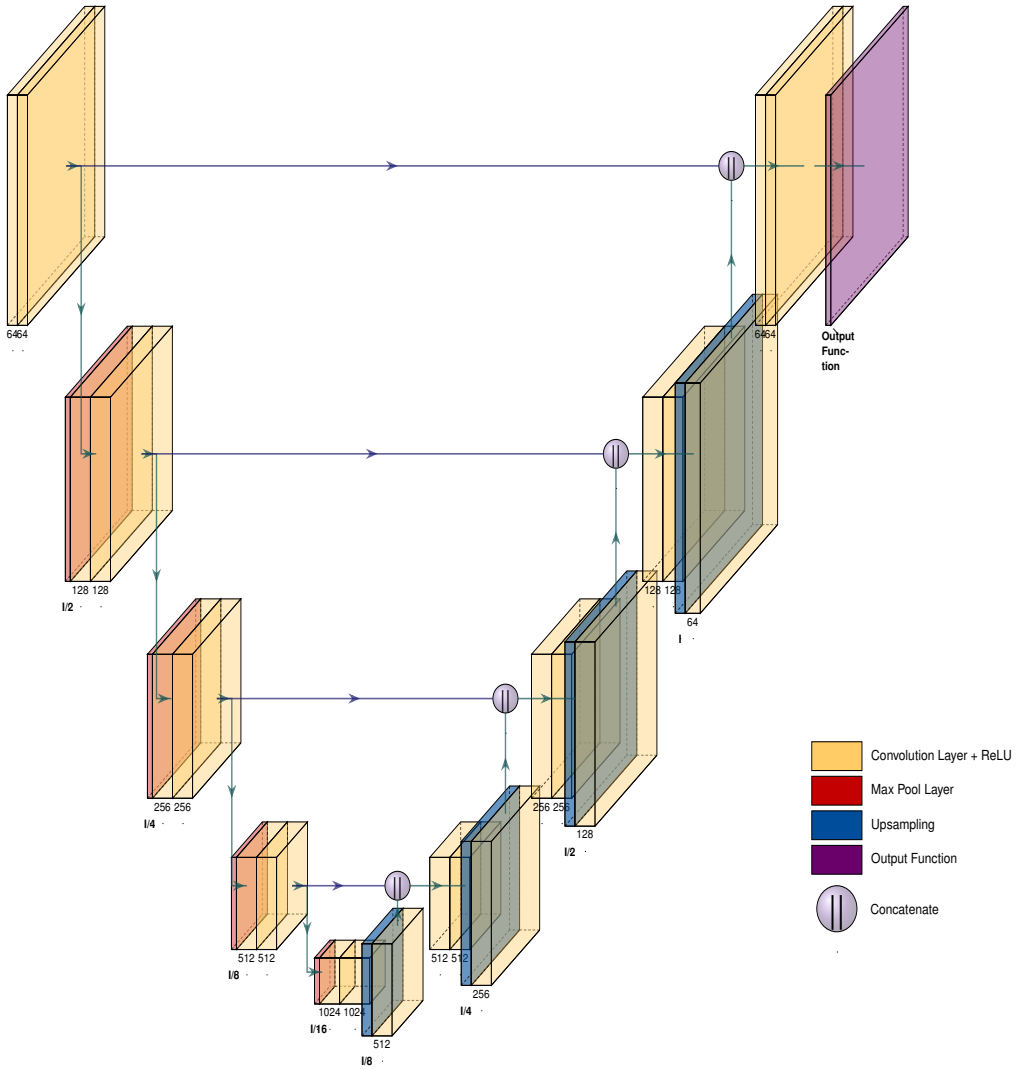


Figure 3.1: U-Net Architecture. **Bold** numbers indicate changes of tensor size by max pool layers and upsampling, with the numbers describing the post-execution tensor size relative to the model input tensor size. Non-bold numbers indicate the number of output channels of convolutional layers. Blue arrows depict skip connections. Green arrows depict normal connections. Based on PlotNeuralNet [32].

3.2.3 Dilated Convolutional Layer

Dilated convolutional layers, also called atrous convolutional layers, are a modification of classic convolutional layers, adding the dilation parameter. They aim to increase the receptive field, i.e., how much of the image the kernel can see, without increasing the kernel size. The dilation parameter acts similaly to the stride parameter. However, instead of being applied after each kernel application, it is applied when selecting which pixels the kernel is applied to. If the dilation is 1, it is simply a normal convolutional layer. However, if the dilation is 2, the layer will select only every second pixel in both the x- and y-directions (see Figure 3.3).

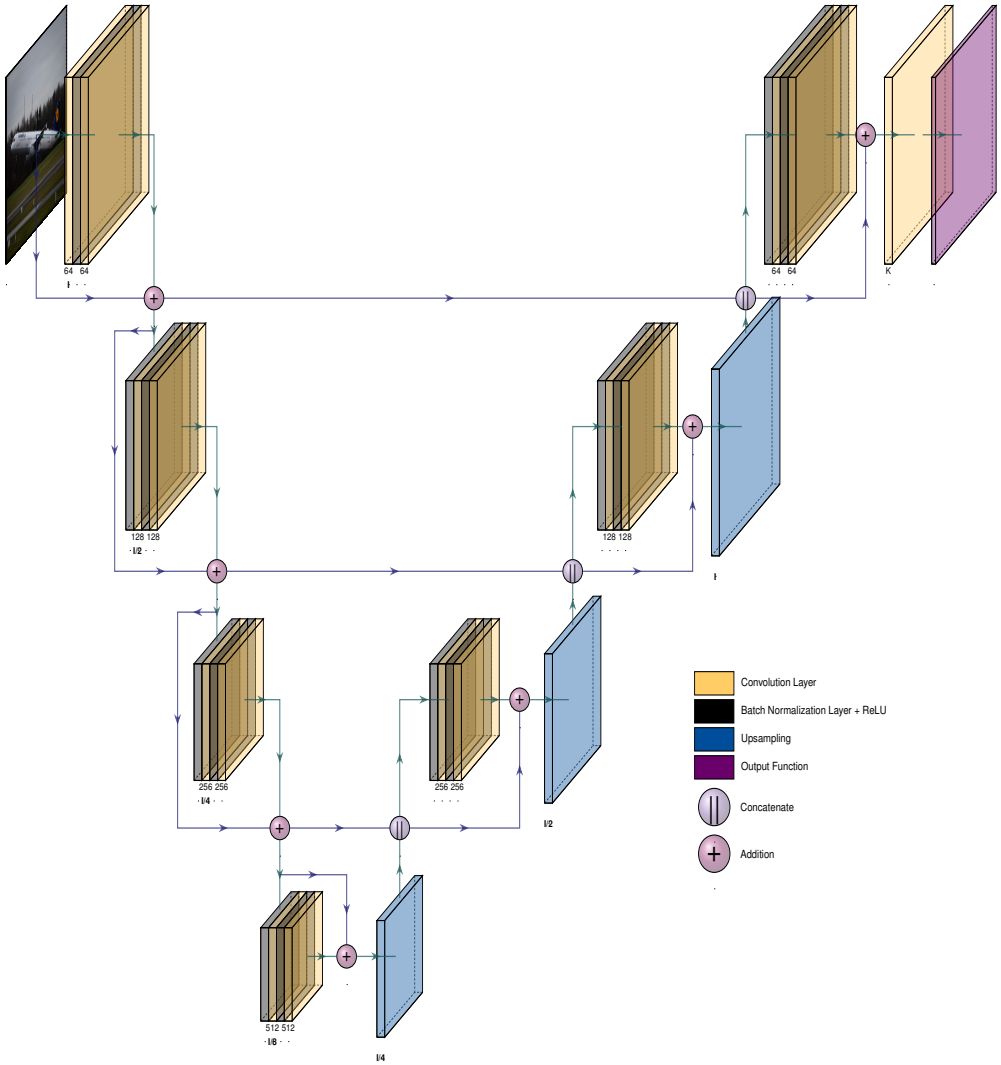


Figure 3.2: Deep ResUnet Architecture. **Bold** numbers indicate changes of tensor size by convolutional layers and upsampling, with the numbers describing the post-execution tensor size relative to the model input tensor size. Non-bold numbers indicate the number of output channels of convolutional layers. Blue arrows depict skip connections. Green arrows depict normal connections. Based on PlotNeuralNet [32].

3.2.4 AtrousDenseUDeconvNet

Su, Li, Ma, *et al.* [11] extended U-Net by using dense blocks containing dilated convolutional layers to pre-process the data, and transposed convolutional layers to post-process it. A dense block consists of three dilated convolutional layers with dilations of 2, 5, and 8, respectively. These dilated convolutional layers are used as inputs for each other, and all of their outputs are concatenated together, with the output of the previous block as the input for the subsequent block. The architecture also utilizes transition layers, which consist of a convolutional layer, an average pooling layer with kernel size 2, as well as a batch normalization layer.

The network starts its processing with a convolutional layer, followed by a dense block. This is followed by a transition layer, feeding into another dense block. After passing through a second transition layer, the data is then used as the input for a modified U-Net

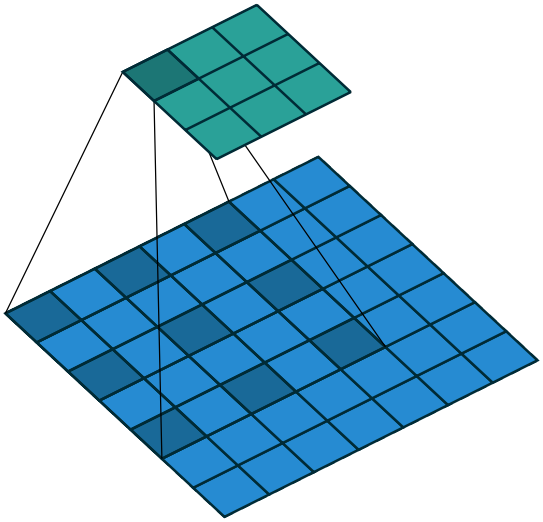


Figure 3.3: Dilated Convolutional Layer with kernel size 3 and dilation 2. The lower blue grid shows the input image for the example of the first output pixel. The dark blue tiles are used to calculate the output value. The upper teal grid represents the layer output. Source: Dumoulin and Visin [34].

architecture. Contrary to the base architecture, it uses only one convolutional layer per level in the contracting and the expanding path. However, it adds dropout layers after each level in the contracting path. Unlike the Deep ResUnet, it keeps the convolutional layers after the upsampling; however, consistently with it, the AtrousDenseUDeconvNet utilizes padding in its convolutional layers. Su, Li, Ma, *et al.* [11], however, specify that "the size of all the convolutional kernels was 3 [. . .]", whereas Ronneberger, Fischer, and Brox [7] use kernel size of 2 for the "up-convolution" convolutional layers. The output of the modified U-Net is followed by four transposed convolutional layers. Transposed convolutional layers are modified convolutional layers with their forward and backward passes swapped. Su, Li, Ma, *et al.* [11] measured a significant improvement in their metrics for the semantic segmentation of the Potsdam dataset using the AtrousDenseUDeconvNet compared to a standard U-Net.

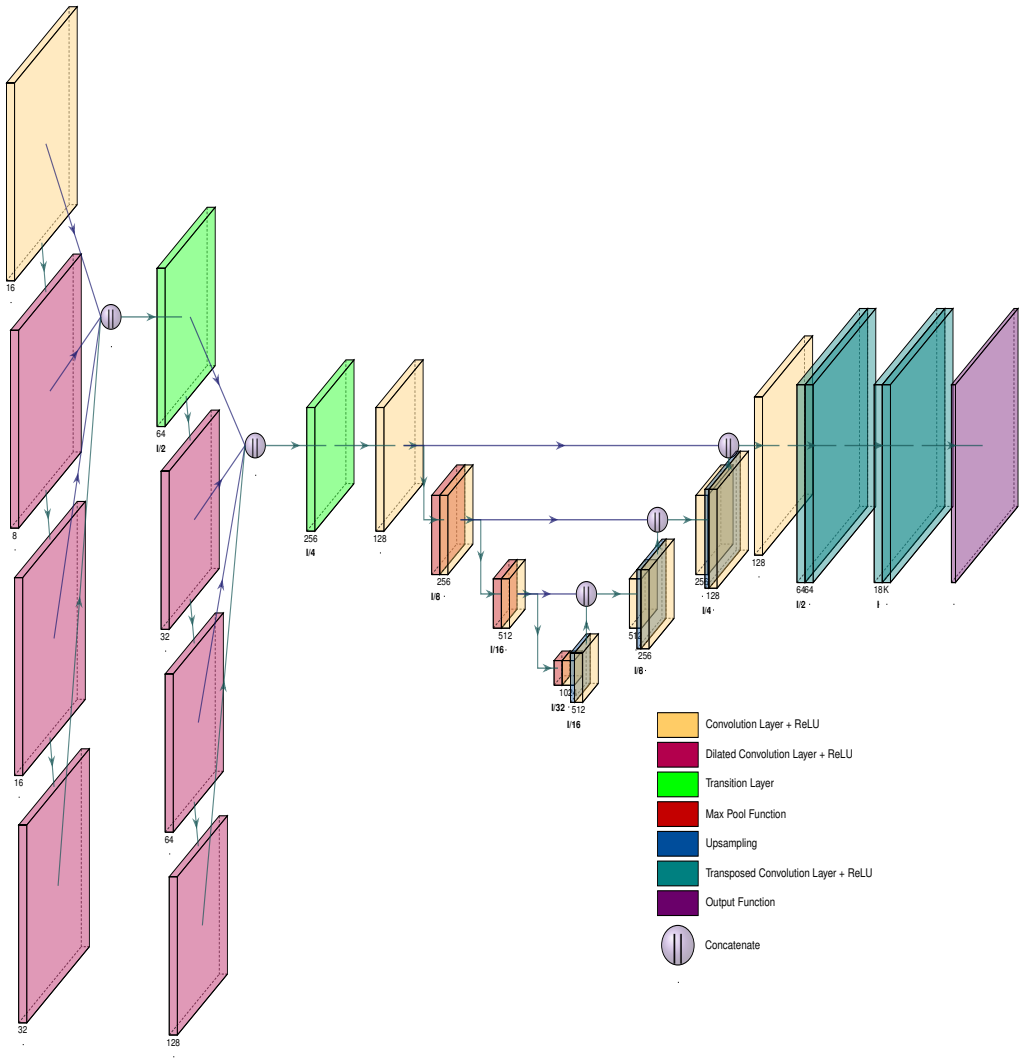


Figure 3.4: AtrousDenseUDeconvNet Architecture. **Bold** numbers indicate changes of tensor size by average pool layers, max pool layers, upsampling, and transposed convolutional layers, with the numbers describing the post-execution tensor size relative to the model input tensor size. Non-bold numbers indicate the number of output channels of convolutional layers. Blue arrows depict skip connections. Green arrows depict normal connections. Based on PlotNeuralNet [32].

4 Experiments

The hardware used in the experiments is an AMD Ryzen 9 5900X processor with 12 cores, 24 threads, and 32 GB of RAM. The models were trained on an AMD Radeon RX 6800 XT GPU with 16 GB of VRAM. The models were run on Ubuntu 22.04, Python 3.10, and PyTorch 2.8.0 for AMD ROCm 6.4.

4.1 Metrics

4.1.1 Mean Intersection over Union

The Intersection over Union (IoU) is a typical metric in image segmentation applications to evaluate how similar the model output is to the ground truth. For binary classification applications, it is calculated by taking the True Positive (TP) and dividing it by the sum of the TP, the False Negative (FN), and the False Positive (FP), as shown in Equation 4.1:

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}. \tag{4.1}$$

For non-binary classification, the IoUs are calculated separately for each class, and then their mean is taken to determine the mean Intersection over Union (mIoU).

4.1.2 Expected Calibration Error

In theory, measuring calibration performance using Equation 3.5 would be optimal. However, since the probability distribution is in practice not known, it must be approximated. To approximate it, the model is evaluated on a part of the dataset, and its output is grouped into bins based on the confidence of the model. Each bin keeps track of the the sum of the confidences assigned to it, as well as if the pixels assigned to it were predicted correctly. The number of bins is a hyperparameter, and different binning strategies exist. The basic approach is to evenly split up the range of probabilities according to the number of bins, and then assign each pixel confidence into its corresponding bin. Using the bins, the miscalibration of the model can then be quantified by the Expected Calibration Error (ECE)[35]. The metric computes the mean confidence, as well as the relative frequency of the positive class for binary problems, or accuracy for non-binary problems, for all bins. It then takes the absolute value of the difference between these two values and multiplies it by the relative number of samples in the bin, i.e.:

$$\text{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|, \tag{4.2}$$

where B is the number of bins, n_b is the number of data points in bin b, N is the total number of data points, and $\text{acc}(b)$ and $\text{conf}(b)$ are the accuracy and mean confidence of bin b, respectively.

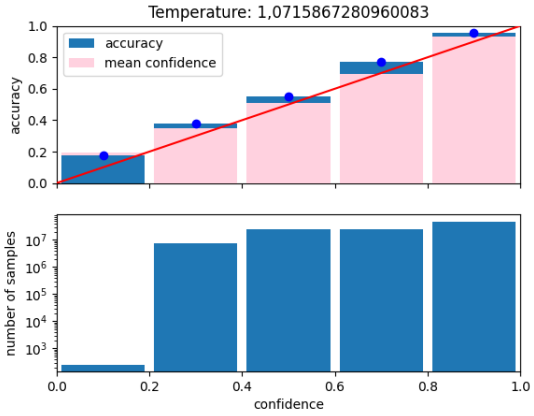


Figure 4.1: The top diagram shows a reliability diagram. The blue and pink bars show accuracy and mean confidence of the bin, respectively. The red diagonal shows the identity, signifying ideal calibration. The bottom diagram shows the sample distribution, giving the number of samples per bin on a logarithmic scale.

4.1.3 Reliability Diagram

While ECE and Static Calibration Error (SCE) are useful to quantify the miscalibration, they can be difficult to interpret. The reliability diagram, based on DeGroot and Fienberg [36], is the standard visualization of calibration performance. It is laid out as a vertical bar chart, showing two bars for each bin. The first bar shows the mean confidence of all the elements assigned to the bin, with the second bar showing how accurate the predictions were as a comparison. Binary classification problems show the relative frequency of the positive class for comparison. However, for the non-binary classification problems used in this paper, the accuracy is plotted. For well calibrated networks, the bars should be of similar height, showing that the predicted accuracy of the model roughly equals the measured accuracy. Both bars should also follow the red diagonal, which depicts the theoretical optimal calibration (See Figure 4.1).

4.2 Datasets

4.2.1 Potsdam Dataset

The Potsdam semantic labeling dataset was released by the International Society for Photogrammetry and Remote Sensing in connection with a 2015 contest [37]. It contains Earth observation imagery of Potsdam, Germany, consisting of 38 patches, each with a resolution of 6000×6000. The images are available in the following combinations of Infrared (IR), Red (R), Green (G), and Blue (B): IR-R-G, R-G-B, R-G-B-IR. In this work, the R-G-B version of the images are used. The ground truth of the dataset classifies the patches into six classes: impervious surfaces, buildings, low vegetation, trees, cars, and background. The Ground Sampling Distance (GSD) of the dataset is 5 cm.

	Postdam	Data Fusion
Original Resolution	6000×6000	512×512
Number of Input Images	38	4333
Train Images	23	2600
Validation Images	7	866
Test Images	8	867
Number of samples	200	5
Sample Resolution	256×256	256×256

Table 4.1: Dataset pre-processing details per dataset

4.2.2 2025 Data Fusion Dataset

The IEEE Geoscience and Remote Sensing Society (GRSS) held the 2025 Data Fusion Contest with the topics "All-Weather Land Cover" and "Building Damage Mapping". The "All-Weather Land Cover" dataset [38], contains pairs of optical and SAR images, which are used to predict the land coverage independent of weather conditions. The GRSS intend for the participants to use both optical and SAR images in training, since optical images can suffer from significant occlusions (e.g., clouds and smoke). Conversely, because SAR imagery operates in a lower frequency spectrum, its detection capabilities are unaffected by most optical occlusions, allowing it to operate in a variety of weather conditions. However, training solely on SAR images provides less information than combining both modalities. While the models participating in the contest are intended to use a combination of optical and SAR images in training and use SAR images during inference, the goal of this thesis is not to research data fusion. Thus, in this thesis the same data types are used during training and inference. The ground truth labels contain eight classes: bareland, rangeland, developed space, road, tree, water, agriculture land, and building. Because the dataset contains images from multiple sources, they do not all have the same GSDs, with it ranging from 15 to 50 cm.

4.2.3 Dataset Pre-Processing

The input images from the datasets were split up into a training set (60%), validation set (20%), and a test set (20%). For data augmentation, multiple 256× 256 images were extracted from random segments of each input image and its corresponding ground truth; afterwards, they were mirrored horizontally and vertically, each with a probability of 50%. The number of input images in each sub-dataset, the number of samples per input image, and the sample resolution are summarized in Table 4.1.

4.3 Experiment 1: Accuracy and Uncertainty Quantification

For each dataset and model architecture, a model was using both SoftMax and Sigmoid, resulting in 18 total models. Each model was trained for 20 epochs with a learning rate of 10^{-4}. All models used a batch size of 16. The training used AdamW as the optimizer, cross-entropy loss for the SoftMax models, and binary cross-entropy loss for the Sigmoid models. For the AtrousDenseUDeconvNet models, a dropout probability of 0.1 was chosen, since Su, Li, Ma, *et al.* [11] simply states that they "used consistent and appropriate dropout".
In the first experiment, the baseline results of the SoftMax and Sigmoid models with regards to accuracy and uncertainty quantification are collected and compared. To this end, quantifiable metrics, as well as reliability diagrams, are contrasted to identify differences in their results. For the reliability diagrams, 15 was chosen as the number of bins, balancing the granularity to be able to show miscalibration, with each bin containing enough samples to be able to accurately approximate its calibration.

4.4 Experiment 2: Scalar Temperature Scaling vs Vector Temperature Scaling

In the second experiment, scalar temperature scaling and vector temperature scaling are contrasted. Specifically, it is investigated whether vector temperature scaling allows Sigmoid models to achieve improved calibration.
For this, each model is calibrated by locking all model parameters and adding the temperature as an additional parameter. The temperature is then trained on the validation set for 50 epochs, using the ECE as its loss function, and Stochastic Gradient Descent as its optimizer. The loss is backpropagated every epoch for the models trained on the Potsdam dataset, and every 1000 iterations for the other models, due to the larger size of the Data Fusion dataset leading to memory constraints.

5 Results

5.1 Accuracy

The results show that, throughout all datasets and and both output functions, AtrousDenseUDeconvNet achieves the highest values for mIoU and overall accuracy.
For the Potsdam dataset, all Sigmoid models show slightly better overall multi-class metrics, i.e., mIoU and overall accuracy, than their SoftMax counterparts. However, the class accuracy shows trends where all SoftMax models have higher accuracy for impervious surfaces, low vegetation and cars and all Sigmoid models have higher accuracy for buildings. Trees is the only class where a model has higher accuracy and a different model has lower accuracy, between SoftMax and Sigmoid.
For the models that were trained on the Data Fusion SAR images, U-Net shows almost identical multi-class metrics, with slight differences in the class accuracies. For Deep ResUnet and AtrousDenseUDeconvNet, the SoftMax models show slightly higher multi-class metrics, while the class accuracies contain larger differences between SoftMax and Sigmoid compared to U-Net. For the Data Fusion SAR models however, there appears to be no trend in the class accuracy differences between SoftMax and Sigmoid.
The models trained on the Data Fusion RGB images show mixed results in the multi-class metrics with U-Net and Deep ResUnet showing better results when trained with SoftMax and AtrousDenseUDeconvNet performing better using Sigmoid. For the class accuracies there are some trends between SoftMax and Sigmoid with the SoftMax models outperforming in the bareland, road, tree, and water classes. The Sigmoid models show higher accuracies in the developed space, and agriculture land classes.

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
mIoU	0.517	0.581	0.605	
Accuracy	0.722	0.752	0.797	
Class Accuracy				
Impervious Surfaces	0.831	0.767	0.870	0.349
Buildings	0.725	0.742	0.859	0.371
Low vegetation	0.832	0.859	0.813	0.142
Trees	0.254	0.590	0.354	0.116
Cars	0.716	0.830	0.817	0.023

Table 5.1: Accuracy metrics of models using SoftMax on the Potsdam dataset

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
mIoU	0.542	0.590	0.613	
Accuracy	0.750	0.763	0.782	
Class Accuracy				
Impervious Surfaces	0.789	0.737	0.692	0.349
Buildings	0.866	0.847	0.939	0.371
Low vegetation	0.642	0.807	0.665	0.142
Trees	0.410	0.514	0.708	0.116
Cars	0.670	0.812	0.741	0.023

Table 5.2: Accuracy metrics of models using Sigmoid on the Potsdam dataset

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
mIoU	0.253	0.282	0.296	
Accuracy	0.461	0.485	0.511	
Class Accuracy				
Bareland	0.000	0.000	0.000	0.015
Rangeland	0.497	0.453	0.516	0.236
Developed Space	0.485	0.522	0.595	0.228
Road	0.000	0.006	0.008	0.076
Tree	0.559	0.554	0.648	0.141
Water	0.492	0.684	0.672	0.118
Agriculture Land	0.511	0.639	0.437	0.067
Building	0.518	0.476	0.445	0.117

Table 5.3: Accuracy metrics of models using SoftMax on the Data Fusion SAR dataset

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
mIoU	0.254	0.276	0.283	
Accuracy	0.460	0.471	0.497	
Class Accuracy				
Bareland	0.000	0.012	0.000	0.015
Rangeland	0.489	0.610	0.598	0.236
Developed Space	0.448	0.476	0.594	0.228
Road	0.000	0.065	0.000	0.076
Tree	0.528	0.385	0.486	0.141
Water	0.539	0.674	0.646	0.118
Agriculture Land	0.517	0.584	0.522	0.067
Building	0.584	0.338	0.340	0.117

Table 5.4: Accuracy metrics of models using Sigmoid on the Data Fusion SAR dataset

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
mIoU	0.450	0.491	0.531	
Accuracy	0.666	0.710	0.738	
Class Accuracy				
Bareland	0.163	0.046	0.139	0.017
Rangeland	0.749	0.721	0.764	0.238
Developed Space	0.592	0.731	0.685	0.228
Road	0.284	0.570	0.610	0.074
Tree	0.800	0.864	0.875	0.142
Water	0.835	0.854	0.880	0.116
Agriculture Land	0.517	0.531	0.580	0.070
Building	0.712	0.607	0.741	0.114

Table 5.5: Accuracy metrics of models using SoftMax on the Data Fusion RGB dataset

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
mIoU	0.428	0.473	0.539	
Accuracy	0.667	0.703	0.753	
Class Accuracy				
Bareland	0.002	0.045	0.065	0.017
Rangeland	0.784	0.677	0.792	0.238
Developed Space	0.770	0.821	0.746	0.228
Road	0.178	0.259	0.515	0.074
Tree	0.693	0.848	0.855	0.142
Water	0.815	0.854	0.864	0.116
Agriculture Land	0.565	0.555	0.630	0.070
Building	0.516	0.663	0.773	0.114

Table 5.6: Accuracy metrics of models using Sigmoid on the Data Fusion RGB dataset

5.2 Uncertainty

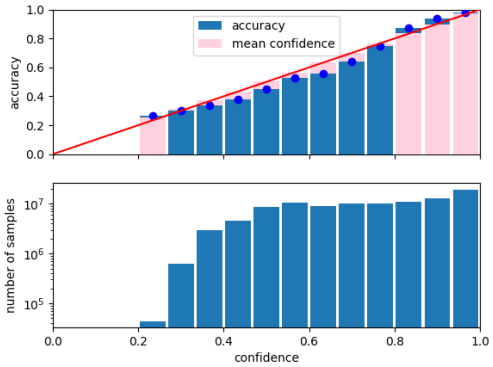


Figure 5.1: Reliability diagram for the uncalibrated U-Net using SoftMax on the Potsdam dataset

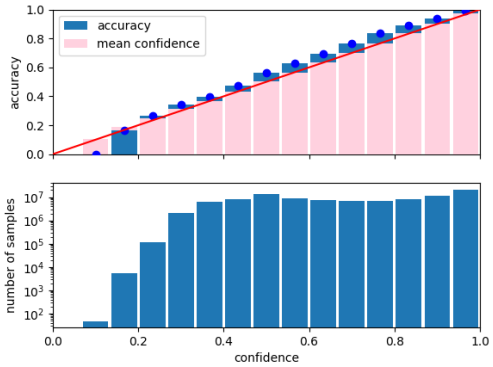


Figure 5.2: Reliability diagram for the uncalibrated U-Net using Sigmoid on the Potsdam dataset

The U-Net model that was trained on the Potsdam dataset using the SoftMax function shows overconfidence in its predictions, i.e., the confidence in its predictions is higher than their accuracy, in the range of the 6th bin to the 12 bin, i.e., between 33.3% and 80% confidence. In the 13th and 14th bin, i.e., between 80% and 93.3% confidence, it shows slight underconfidence, i.e. the accuracy of its predictions is higher than the model's confidence. The model predictions mostly get grouped into the bins following the 6th bin (33.3%), with very few predictions below the 5th bin (26.6%).

The Sigmoid model shows varying degrees of underconfidence in its predictions, starting at the 4th bin (20%). Most of the model's predictions are grouped into bins starting with the 5th bin (26.6%) with few predictions below the 4th bin (20%).

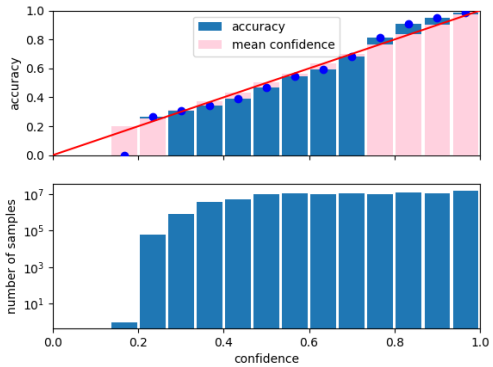


Figure 5.3: Reliability diagram for the scalar temperature scaled U-Net using SoftMax on the Potsdam dataset

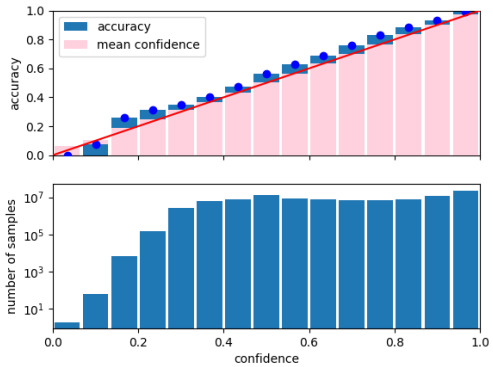


Figure 5.4: Reliability diagram for the scalar temperature scaled U-Net using Sigmoid on the Potsdam dataset

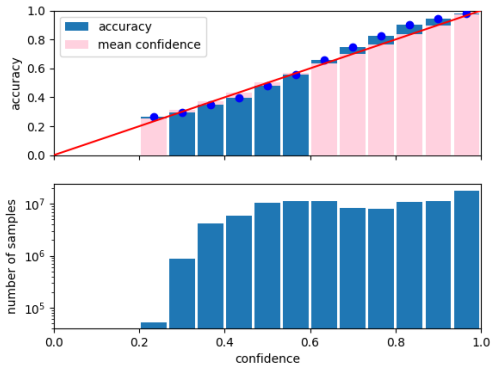


Figure 5.5: Reliability diagram for the vector temperature scaled U-Net using SoftMax on the Potsdam dataset

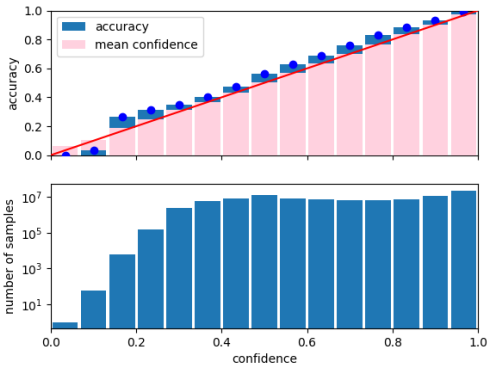


Figure 5.6: Reliability diagram for the vector temperature scaled U-Net using Sigmoid on the Potsdam dataset

The two methods of temperature scaling both show an improvement in the calibration of the U-Net that was trained using SoftMax, with the vector temperature scaling showing superior calibration compared to the scalar temperature scaling.

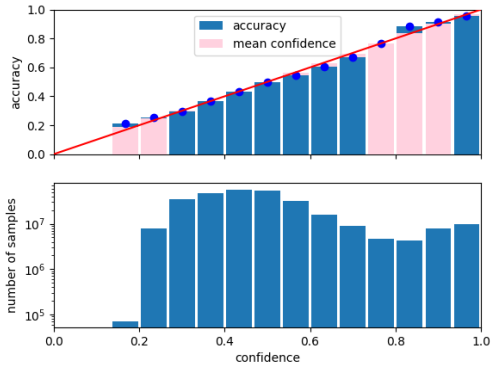


Figure 5.7: Reliability diagram for the vector temperatued scaled Deep ResUnet using SoftMax on the Data Fusion SAR dataset

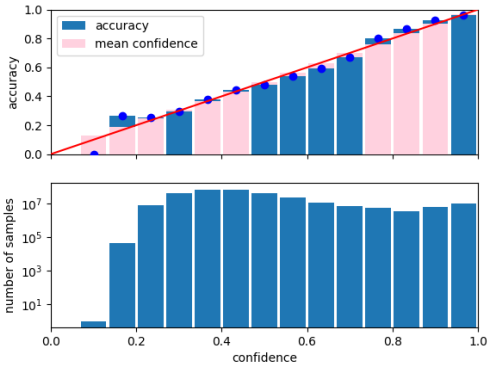


Figure 5.8: Reliability diagram for the vector temperatued scaled Deep ResUnet using Sigmoid on the Data Fusion SAR dataset

The Deep ResUnet model that was trained on the Data Fusion SAR dataset using the SoftMax output function and calibrated using vector temperature scaling shows low calibration errors. Its sample distribution shows a high amount of data between the 5th bin and the 9th bin, i.e., between 26.6% and 60% confidence.

The Deep ResUnet model that was trained on the Data Fusion SAR dataset using the Sigmoid output function shows slight overconfidence between the 8th bin and the 11th bin, i.e., between 46.6% and 73.3% confidence. It also shows slight underconfidence between the 12th bin and the 14th bin, i.e., between 73.3% and 93.3% confidence. The model shows a slightly higher number of samples between the 5th bin and the 9th bin, i.e., be-

tween 26.6% and 60% confidence, compared to beyond the 9th bin.

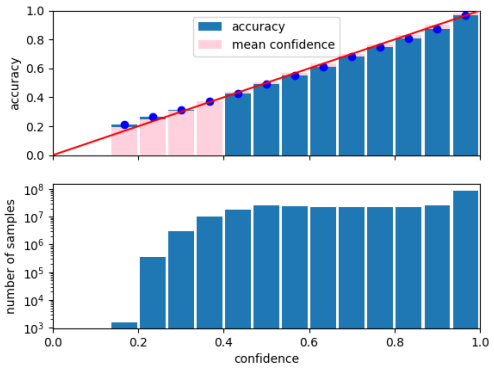


Figure 5.9: Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using SoftMax on the Data Fusion RGB dataset

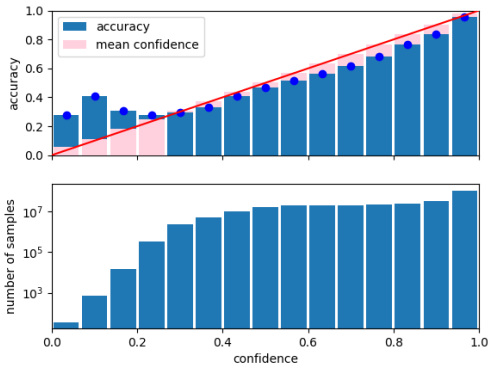


Figure 5.10: Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using Sigmoid on the Data Fusion RGB dataset

The AtrousDenseUDeconvNet model that was trained on the Data Fusion RGB dataset using the SoftMax output function shows very low calibration error in all bins that contain samples.
The AtrousDenseUDeconvNet model that was trained on the Data Fusion RGB dataset using the Sigmoid output function shows increasing amounts of overconfidence starting at the 6th bin (33.3%).
The remaining reliability diagrams, the used temperatures, and confusion matrices for all models can be found in the Appendix in Section 6.

5.3 Discussion

The multi-class metrics show no consistent differences between models trained using SoftMax and Sigmoid. While the class accuracy shows some trends on the Potsdam and Data Fusion RGB datasets, they are too inconsistent to draw the conclusion that one training method is superior with regard to accuracy. This result is desirable since this work is trying to investigate whether using Sigmoid as the output function leads to improved uncertainty quantification; therefore, the result that using it is at least equivalent would allow for its usage in practice, if it shows superior uncertainty quantification.

Su, Li, Ma, *et al.* [11] also trained a U-Net model, as well as their AtrousDenseUDeconvNet, on the Potsdam dataset. The accuracy of the U-Net model trained in this work and their model is comparable; however, their AtrousDenseUDeconvNet model is reported to have significantly higher mIoU and accuracy values. The class accuracies also show significant differences, with the models using SoftMax trained in this work showing significantly higher accuracies on the impervious surfaces, low vegetation, and cars classes.

	U-Net	AtrousDenseUDeconvNet
mIoU	0.525	0.660
Accuracy	0.717	0.828
Class Accuracy		
Impervious Surfaces	0.268	0.556
Buildings	0.807	0.899
Low vegetation	0.718	0.770
Trees	0.870	0.845
Cars	0.658	0.760

Table 5.7: Accuracy metrics of models trained by Su, Li, Ma, *et al.* [11] on the Potsdam dataset

	yhZhou	Good_Lin	Nazira
mIoU	0.481	0.468	0.455
Class Accuracy			
Bareland	0.039	0.039	0.036
Rangeland	0.376	0.331	0.340
Developed Space	0.376	0.379	0.339
Road	0.437	0.460	0.408
Tree	0.558	0.502	0.519
Water	0.822	0.847	0.803
Agriculture Land	0.649	0.621	0.639
Building	0.591	0.563	0.555

Table 5.8: Accuracy metrics of models trained by the top 3 participants in the Data Fusion contest.

While, their models show a significantly higher accuracy for the trees class, as well as a higher accuracy on the buildings class. The difference in results is most likely explained by differences in methodology. Su, Li, Ma, *et al.* [11] used the IR-R-G (infrared, red, green) images of the Potsdam dataset instead of the R-G-B (red, green, blue) images used in this work. They also transformed the labels of the dataset from R-G-B values to grayscale values, as well as employing a higher amount of data augmentation functions.
Compared to the models participating in the Potsdam semantic labeling contest, the models trained in this work show a similar overall accuracy, compared to some of the models, with most of the submitted models showing a significantly higher accuracy of 85-90%. This is likely due the methodology employed in this work not being focused on optimizing the model performance, but comparing the performance of different kinds of models, and due to this pre-processing the datasets less than one would to train a model for a contest.

Compared to the models of the top participants in the Data Fusion Contest, the models trained in this work on the Data Fusion SAR dataset achieve significantly worse results.

The models that were trained on the Data Fusion RGB dataset achieve similar to better results. This is due to the methodology used; the models trained in this work were trained either on the SAR images or the RGB images and then also tested on the same. However, due to it being a Data Fusion contest, the models of the participants were trained on both the RGB as well as the SAR images, and then tested only on the SAR images, making the results not comparable.

Some of the models using SoftMax as their output function show calibration errors, especially when they are uncalibrated. Their calibration improves after calibration, using either scalar temperature scaling or vector temperature scaling. Some models that were trained using SoftMax (See Figure 5.9) are able to output well calibrated predictions without being calibrated.

The models that were trained using Sigmoid similarly show both good results, in some models, even when uncalibrated (See Figure 5.12). However, there are also some models showing similar calibration errors to the SoftMax models (See Figure 5.10).

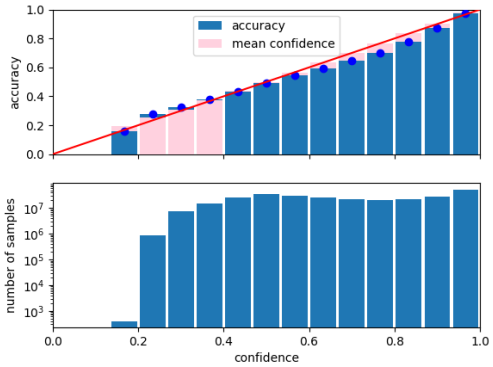


Figure 5.11: Reliability diagram for the uncalibrated U-Net using SoftMax on the Data Fusion RGB dataset

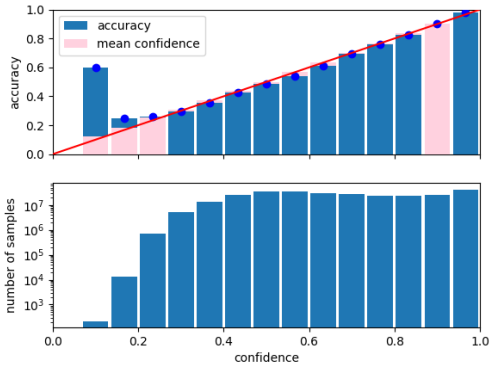


Figure 5.12: Reliability diagram for the uncalibrated U-Net using Sigmoid on the Data Fusion RGB dataset

Compared to the models that were trained using SoftMax, the models that were trained using Sigmoid show significantly higher sample sizes in low confidence bins. This can be directly attributed to the Sigmoid function calculating the confidence for each class separately and thus making it much more likely for the model to output low confidence samples. The ability to output low confidence could provide a significant advantage to models trained using Sigmoid in practice.

The calibration using vector temperature scaling shows better calibration results compared to scalar temperature scaling. However, vector temperature scaling also shows a potential limitation in the calibration methodology. Since the temperatures are determined as a parameter of the network using Stochastic Gradient Descent, the temperature of some classes barely change from the start value, while this could be the result of good calibra-

tion of those classes, it is likely often the result of their low sample size. This results in some classes not being calibrated, which would lead to issues if used in practice.
Across all models and datasets, it appears that the models that were trained using SoftMax provide quite reasonably calibrated uncertainty quantification. Additionally, the models that were trained using Sigmoid show no consistent advantage in the uncertainty quantification compared to the models that were trained using SoftMax.

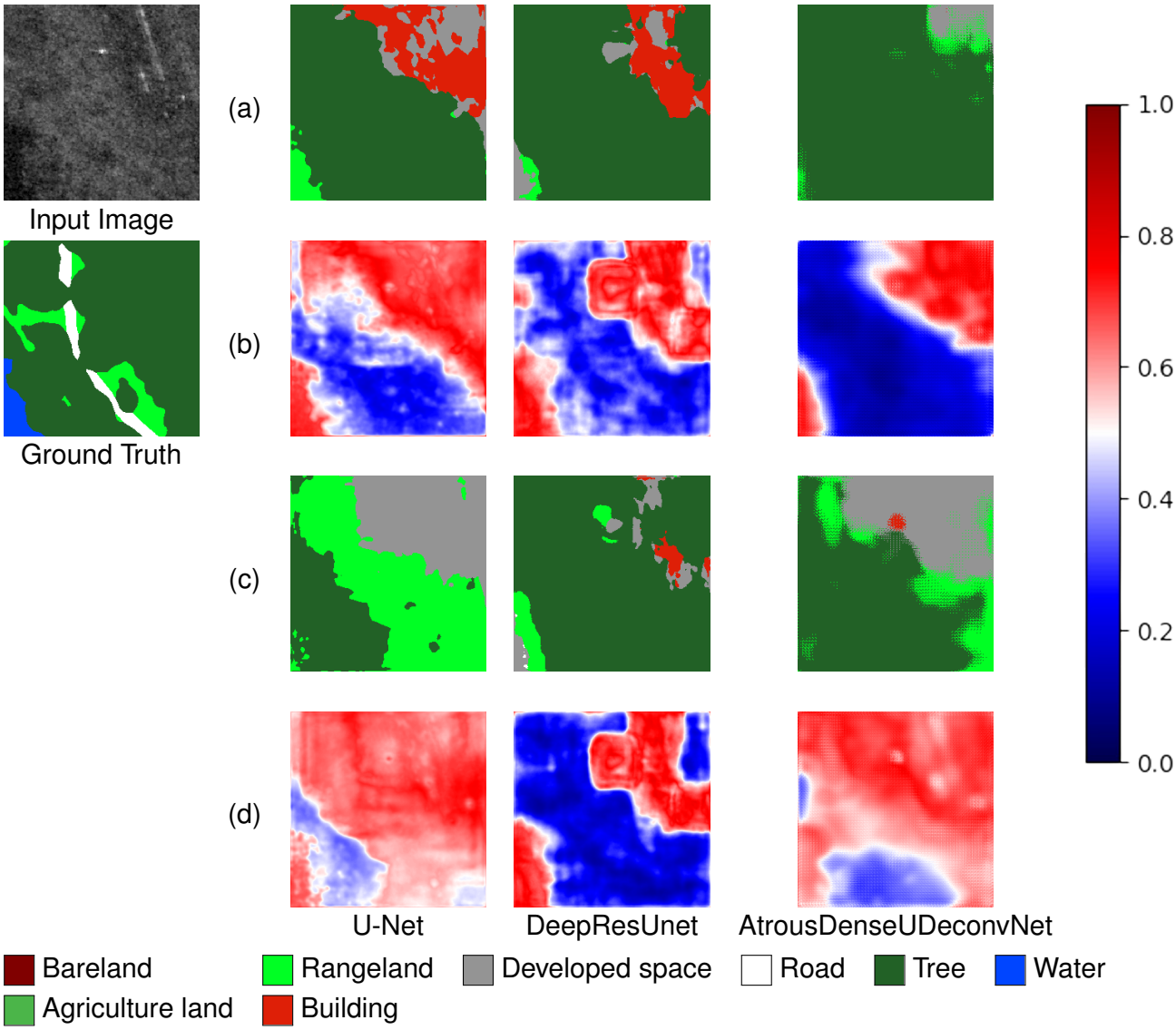


Figure 5.13: Example of the output of models trained on the Data Fusion SAR dataset; (a) class predictions of models trained using SoftMax; (b) uncertainty of models trained using SoftMax; (c) class predictions of models trained using Sigmoid; (d) uncertainty of models trained using Sigmoid

The sample image for the Data Fusion SAR dataset shows mostly trees and rangeland, with a road and a small amount of water in the bottom left corner. All models predict the majority of the image to contain either trees or rangeland, with the U-Net and the DeepResUnet model trained using SoftMax predicting an area in the top right of the image to

contain the building class, and the U-Net and the AtrousDenseUDeconvNet trained using Sigmoid predicting the top right of the image to contain developed space. All models except for the U-Net and the AtrousDenseUDeconvNet trained using Sigmoid, show low uncertainty, i.e., high confidence, in the majority of the tree predictions. All models show high uncertainty in predicting the top right of the image, as well as the bottom left.
The class predictions by the AtrousDenseUDeconvNet models show checkerboard artifacts, a well known side effect of the usage of the transposed convolution layers used in the architecture [39]. One of the common strategies to avoid such artifacts is the usage of bilinear interpolation, used in the U-Net and DeepResUnet architectures.
All models show rather rough, large scale predictions of the segmentation classes. However, even in these models that show low mIoU and accuracy, it is observable that they are producing meaningful uncertainty predictions.
The sample image for the Data Fusion RGB dataset shows a body of water on the left side of the image and a land mass toward the top right of the image. The bottom left of the image shows something in the water; however, the ground truth simply classifies the entire body of water as water. All models predict the island in the top right largely correctly, only showing higher amounts of uncertainty around the class edges. All models try to classify the anomaly in the water as a mixture between rangeland and trees, while showing high uncertainty in their predictions. The U-Net models show particularly poor prediction of the body of water, with the one that was trained using SoftMax almost classifying the entire body of water as rangeland. The U-Net that was trained using Sigmoid classifies the body of water mostly correctly as water, but still shows high uncertainty in its predictions.
The DeepResUnet and the AtrousDenseUDeconvNet models that were trained using Sigmoid show significantly lower amounts of uncertainty in the center of the body of water, showing more localized high uncertainty towards the bottom left of the image. Compared to their SoftMax counterparts, that show high uncertainty in the bottom left of the image, but also show uncertainties around 50% for more central areas of the body of water.

The sample image of the Potsdam dataset shows a building in the top left corner, with some low vegetation surrounding it. The bottom left and bottom right corners each show a car. All models classify the sample mostly correctly, with only the U-Net models struggling to correctly classify the cars, either partially or entirely. In general, the models show high uncertainty around class borders. The U-Net and the AtrousDenseUDeconvNet models trained using Sigmoid show larger areas of high uncertainty compared to the models trained using SoftMax.
The samples don't show a clear pattern of models trained using SoftMax or Sigmoid producing better class predictions or higher quality uncertainty quantification.

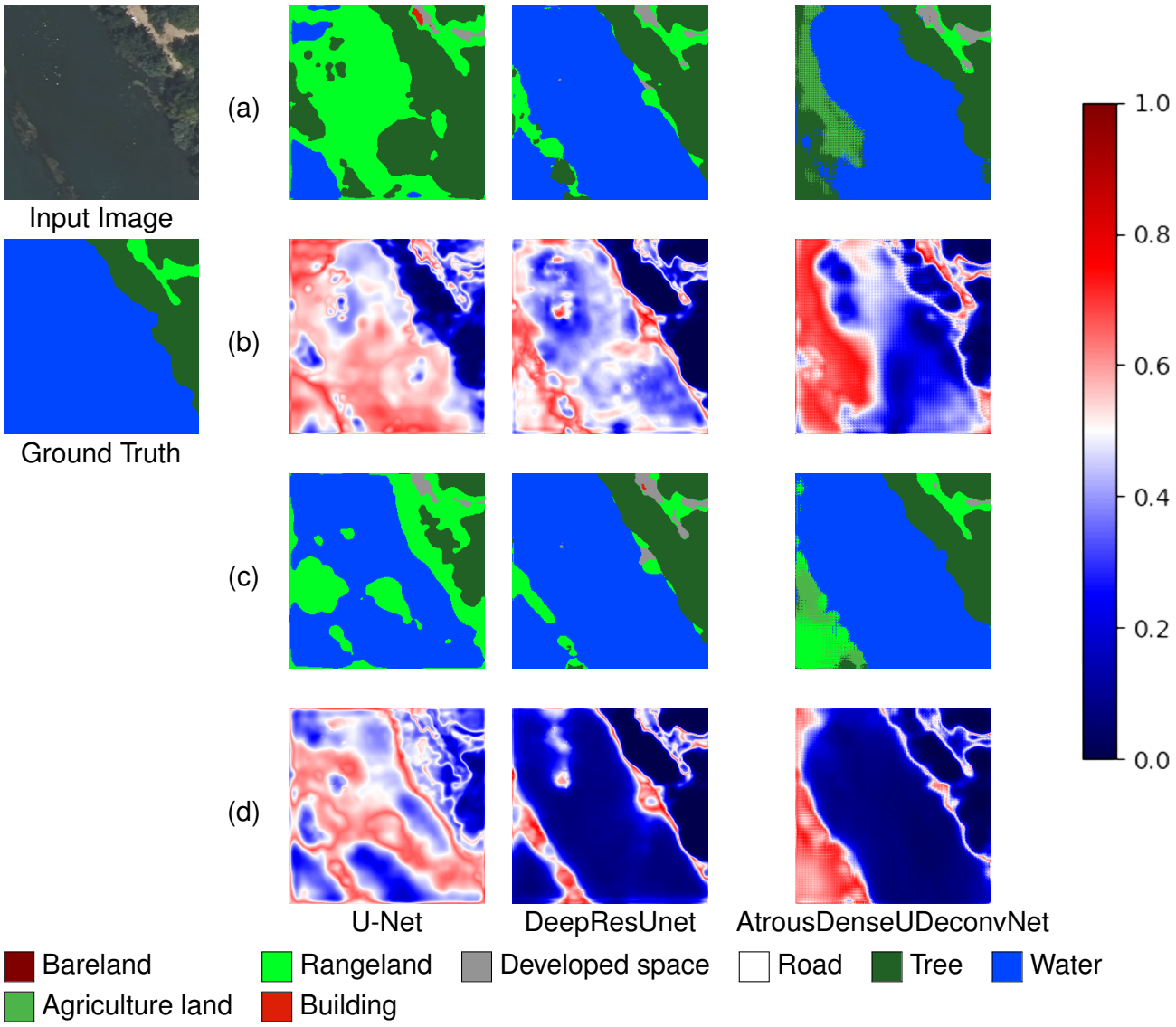


Figure 5.14: Example of the output of models trained on the Data Fusion RGB dataset; (a) class predictions of models trained using SoftMax; (b) uncertainty of models trained using SoftMax; (c) class predictions of models trained using Sigmoid; (d) uncertainty of models trained using Sigmoid

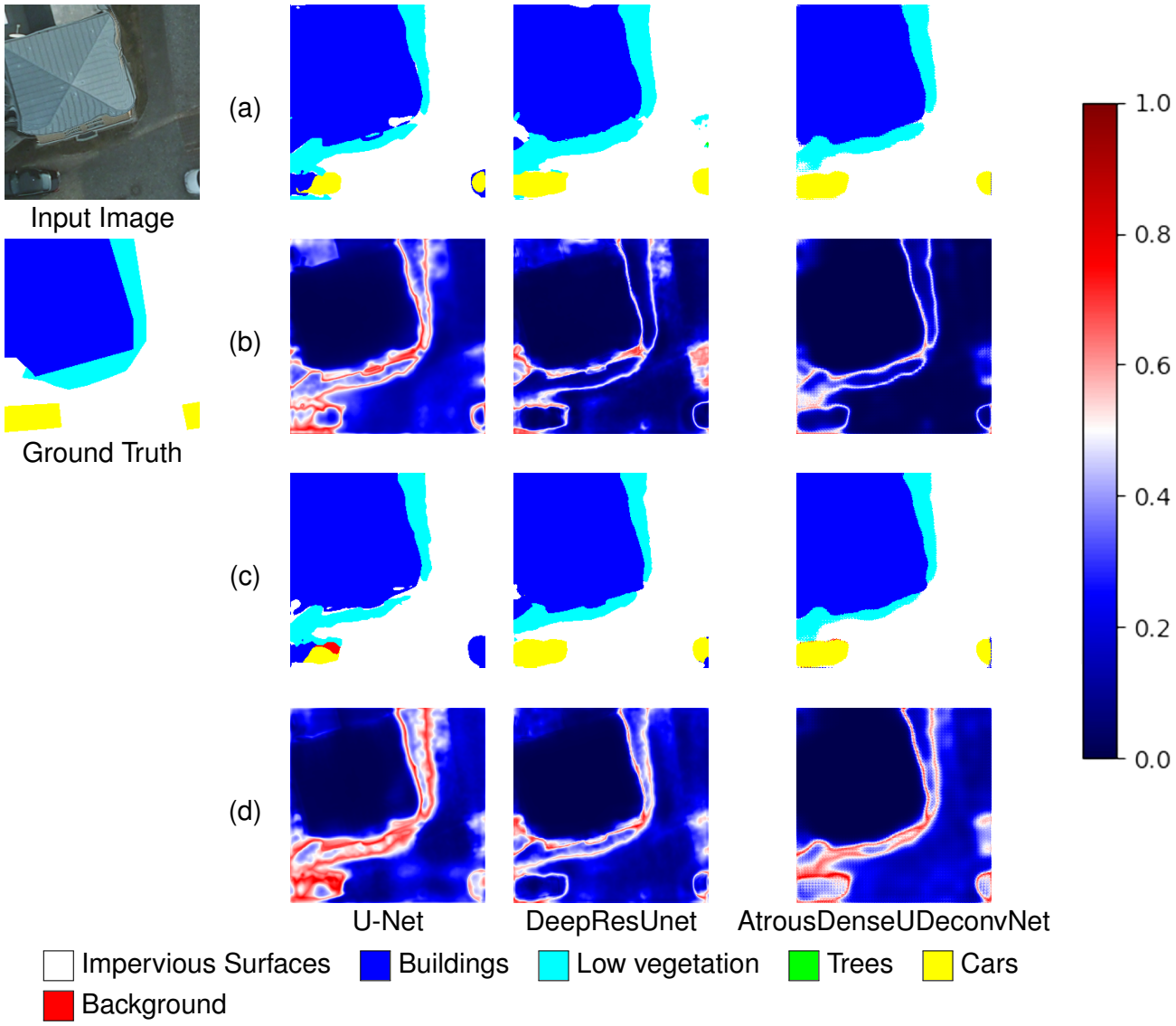


Figure 5.15: Example of the output of models trained on the Potsdam dataset; (a) class predictions of models trained using SoftMax; (b) uncertainty of models trained using SoftMax; (c) class predictions of models trained using Sigmoid; (d) uncertainty of models trained using Sigmoid

6 Conclusion

DNNs have become the state of the art in many fields of research and are used extensively in practical applications. Nevertheless, most research into them aims to maximize averaged accuracy metrics without considering the confidence with which the model produces its outputs. However, research into uncertainty quantification can improve the quality of the results produced by the model even without increasing the model's accuracy. One method of achieving this is by sending samples below a set level of confidence to a human expert for review. This method can lead to far better performance in practice, particularly for well calibrated models [28].

In this work, multiple types of networks were trained on datasets for the semantic segmentation of EOI with the application of land usage analysis. The results show that models trained using SoftMax and Sigmoid produce results that are close in accuracy, with neither method consistently performing better. Regarding uncertainty, some uncalibrated models trained using SoftMax exhibit calibration errors. However, there are also models trained using SoftMax that show reliable uncertainty quantification results, even when uncalibrated, with most models showing good calibration after calibration. The results of models trained using Sigmoid show a significantly higher amount of low-confidence predictions, compared to models trained using SoftMax. However, the results do not consistently show sufficiently distinct calibration quality to conclude that models trained using Sigmoid provide better uncertainty quantification than models trained using SoftMax.

While models trained using Sigmoid do not show an improvement in calibration compared to models trained using SoftMax, their improved ability to output low confidence predictions could suffice to warrant the training using the Sigmoid function, particularly since the results of both output functions regarding accuracy are similar.

In general, the result that models trained using SoftMax can provide reliable uncertainty quantification with little to no post-hoc calibration shows the potential of the usage of uncertainty quantification in a wide range of applications. Since the SoftMax function is already widely used as an output function, models trained using it could be used for uncertainty quantification with minimal effort.

In summary, this work shows that models trained using SoftMax can provide reliable uncertainty quantification results after being calibrated for the land usage analysis of Earth observation imagery. Models trained using Sigmoid show higher amounts of low confidence samples; however, they show no significant improvement in calibration. Users should generally give heightened consideration to actively using the uncertainty quantification provided by models trained using either output function.

References

[1] F. I. Diakogiannis, F. Waldner, P. Caccetta, and C. Wu, "Resunet-a: A deep learning framework for semantic segmentation of remotely sensed data," *ISPRS Journal of Photogrammetry and Remote Sensing*, vol. 162, pp. 94–114, 2020, ISSN: 0924-2716. DOI: https://doi.org/10.1016/j.isprsjprs.2020.01.013. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924271620300149.

[2] Z. Zhang, Q. Liu, and Y. Wang, "Road extraction by deep residual u-net," *IEEE Geoscience and Remote Sensing Letters*, vol. 15, no. 5, pp. 749–753, 2018. DOI: 10.1109/LGRS.2018.2802944.

[3] C. Huang, L. S. Davis, and J. R. G. Townshend, "An assessment of support vector machines for land cover classification," *International Journal of Remote Sensing*, vol. 23, no. 4, pp. 725–749, 2002. DOI: 10.1080/01431160110040323. eprint: https://doi.org/10.1080/01431160110040323. [Online]. Available: https://doi.org/10.1080/01431160110040323.

[4] B. Fröhlich, E. Bach, I. Walde, S. Hese, C. Schmullius, and J. Denzler, "Land cover classification of satellite images using contextual information," *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. II-3/W1, pp. 1–6, 2013. DOI: 10.5194/isprsannals-II-3-W1-1-2013. [Online]. Available: https://isprs-annals.copernicus.org/articles/II-3-W1/1/2013/.

[5] M. Volpi and V. Ferrari, "Semantic segmentation of urban scenes by learning local class interactions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2015, pp. 1–9.

[6] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *CoRR*, vol. abs/1411.4038, 2014. arXiv: 1411.4038. [Online]. Available: http://arxiv.org/abs/1411.4038.

[7] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, N. Navab, J. Hornegger, W. M. Wells, and A. F. Frangi, Eds., Cham: Springer International Publishing, 2015, pp. 234–241, ISBN: 978-3-319-24574-4.

[8] C. Henry, S. M. Azimi, and N. Merkle, "Road segmentation in sar satellite images with deep fully convolutional neural networks," *IEEE Geoscience and Remote Sensing Letters*, vol. 15, no. 12, pp. 1867–1871, 2018. DOI: 10.1109/LGRS.2018.2864342.

[9] E. Nemni, J. Bullock, S. Belabbes, and L. Bromley, "Fully convolutional neural network for rapid flood segmentation in synthetic aperture radar imagery," *Remote Sensing*, vol. 12, no. 16, 2020, ISSN: 2072-4292. DOI: 10.3390/rs12162532. [Online]. Available: https://www.mdpi.com/2072-4292/12/16/2532.

[10] L. Colomba, A. Farasin, S. Monaco, *et al.*, "A dataset for burned area delineation and severity estimation from satellite imagery," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, ser. CIKM '22, Atlanta, GA, USA: Association for Computing Machinery, 2022, pp. 3893–3897, ISBN: 9781450392365. DOI: 10.1145/3511808.3557528. [Online]. Available: https://doi.org/10.1145/3511808.3557528.

[11] Z. Su, W. Li, Z. Ma, and R. Gao, "An improved u-net method for the semantic segmentation of remote sensing images," *Applied Intelligence*, vol. 52, no. 3, pp. 3276–3288, Feb. 2022, ISSN: 1573-7497. DOI: 10.1007/s10489-021-02542-9. [Online]. Available: https://doi.org/10.1007/s10489-021-02542-9.

[12] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17, Sydney, NSW, Australia: JMLR.org, 2017, pp. 1321–1330.

[13] J. Gawlikowski, C. R. N. Tassi, M. Ali, *et al.*, "A survey of uncertainty in deep neural networks," *Artificial Intelligence Review*, vol. 56, no. 1, pp. 1513–1589, Oct. 2023, ISSN: 1573-7462. DOI: 10.1007/s10462-023-10562-9. [Online]. Available: https://doi.org/10.1007/s10462-023-10562-9.

[14] G. Wang, W. Li, M. Aertsen, J. Deprest, S. Ourselin, and T. Vercauteren, "Aleatoric uncertainty estimation with test-time augmentation for medical image segmentation with convolutional neural networks," *Neurocomputing*, vol. 338, pp. 34–45, 2019, ISSN: 0925-2312. DOI: https://doi.org/10.1016/j.neucom.2019.01.103. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231219301961.

[15] J. Denker, D. Schwartz, B. Wittner, S. Solla, R. Howard, and L. Jackel, "Large automatic learning, rule extraction, and generalization," *Complex Systems*, vol. 1, no. 5, pp. 877–922, 1987.

[16] Tishby, Levin, and Solla, "Consistent inference of probabilities in layered networks: Predictions and generalizations," in *International 1989 Joint Conference on Neural Networks*, 1989, 403–409 vol.2. DOI: 10.1109/IJCNN.1989.118274.

[17] W. L. Buntine and A. S. Weigend, "Bayesian back-propagation," *Complex Systems*, vol. 5, no. 6, pp. 603–643, 1991.

[18] L. Hansen and P. Salamon, "Neural network ensembles," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 12, no. 10, pp. 993–1001, 1990. DOI: 10.1109/34.58871.

[19] C. Bai, D. Zhao, M. Zhang, and J. Zhang, "Multimodal information fusion for weather systems and clouds identification from satellite images," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 15, pp. 7333–7345, 2022. DOI: 10.1109/JSTARS.2022.3202246.

[20] L. Yaeger, R. Lyon, and B. Webb, "Effective training of a neural network character classifier for word recognition," *Advances in neural information processing systems*, vol. 9, 1996.

[21] P. Simard, D. Steinkraus, and J. Platt, "Best practices for convolutional neural networks applied to visual document analysis," in *Seventh International Conference on Document Analysis and Recognition, 2003. Proceedings.*, 2003, pp. 958–963. DOI: 10.1109/ICDAR.2003.1227801.

[22] M. Welling and Y. W. Teh, "Bayesian learning via stochastic gradient langevin dynamics," in *Proceedings of the 28th International Conference on International Conference on Machine Learning*, ser. ICML'11, Bellevue, Washington, USA: Omnipress, 2011, pp. 681–688, ISBN: 9781450306195.

[23] G. Hinton, O. Vinyals, and J. Dean, *Distilling the knowledge in a neural network*, 2015. arXiv: 1503.02531 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1503.02531.

[24] Y. Gal and Z. Ghahramani, *Dropout as a bayesian approximation: Representing model uncertainty in deep learning*, 2016. arXiv: 1506.02142 [stat.ML]. [Online]. Available: https://arxiv.org/abs/1506.02142.

[25] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html.

[26] F. Verdoja and V. Kyrki, *Notes on the behavior of mc dropout*, 2021. arXiv: 2008.02627 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2008.02627.

[27] J. Ludwig and R. Hänsch, "Evaluation of post hoc uncertainty quantification approaches for flood detection from sar imagery," *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 18, pp. 15 209–15 221, 2025. DOI: 10.1109/JSTARS.2025.3577690.

[28] J. Haas and B. Rabus, "Uncertainty estimation for deep learning-based segmentation of roads in synthetic aperture radar imagery," *Remote Sensing*, vol. 13, no. 8, 2021, ISSN: 2072-4292. DOI: 10.3390/rs13081472. [Online]. Available: https://www.mdpi.com/2072-4292/13/8/1472.

[29] M. Weiss and P. Tonella, "Uncertainty quantification for deep neural networks: An empirical comparison and usage guidelines," *Software Testing, Verification and Reliability*, vol. 33, no. 6, e1840, 2023, _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/stvr.18 ISSN: 1099-1689. DOI: 10.1002/stvr.1840. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1002/stvr.1840 (visited on 01/07/2026).

[30] M. Możejko, M. Susik, and R. Karczewski, *Inhibited softmax for uncertainty estimation in neural networks*, Apr. 7, 2019. DOI: 10.48550/arXiv.1810.01861. arXiv: 1810.01861[cs]. [Online]. Available: http://arxiv.org/abs/1810.01861 (visited on 01/08/2026).

[31] S. Saito, T. Yamashita, Y. Aoki, T. Yamashita, and Y. Aoki, "Multiple object extraction from aerial imagery with convolutional neural networks," *Electronic Imaging*, vol. 28, pp. 1–9, Feb. 17, 2016, Publisher: Society for Imaging Science and Technology, ISSN: 2470-1173. DOI: 10.2352/ISSN.2470-1173.2016.10.ROBVIS-392. [Online]. Available: https://library.imaging.org/ei/articles/28/10/art00004 (visited on 01/08/2026).

[32] HarisIqbal88, "Plotneuralnet," https://github.com/HarisIqbal88/PlotNeuralNet, 2020.

[33] V. Mnih, "Machine learning for aerial image labeling," https://www.cs.toronto.edu/~vmnih/docs/Mnih_Volodymyr_PhD_Thesis.pdf, Ph.D. dissertation, University of Toronto, 2013.

[34] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," *ArXiv e-prints*, Mar. 2016. eprint: 1603.07285.

[35] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, “Obtaining well calibrated probabilities using bayesian binning,” in *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, ser. AAAI’15, Austin, Texas: AAAI Press, 2015, pp. 2901–2907, ISBN: 0262511290.

[36] M. H. DeGroot and S. E. Fienberg, “The comparison and evaluation of forecasters,” *Journal of the Royal Statistical Society Series D: The Statistician*, vol. 32, no. 1-2, pp. 12–22, Mar. 1983, ISSN: 2515-7884. DOI: 10.2307/2987588. eprint: https://academic.oup.com/jrsssd/article-pdf/32/1-2/12/49920469/jrsssd_32_1-2_12.pdf. [Online]. Available: https://doi.org/10.2307/2987588.

[37] ISPRS, “Potsdam semantic labeling dataset,” https://www.isprs.org/resources/datasets/benchmarks/UrbanSemLab/2d-sem-label-potsdam.aspx, 2015.

[38] J. Xia, H. Chen, C. Broni-Bediako, Y. Wei, J. Song, and N. Yokoya, *Openearthmap-sar: A benchmark synthetic aperture radar dataset for global high-resolution land cover mapping*, 2025. arXiv: 2501.10891 [eess.IV]. [Online]. Available: https://arxiv.org/abs/2501.10891.

[39] A. Odena, V. Dumoulin, and C. Olah, “Deconvolution and checkerboard artifacts,” *Distill*, 2016. DOI: 10.23915/distill.00003. [Online]. Available: http://distill.pub/2016/deconv-checkerboard.

Appendix

Appendix Contents

A Reliability Diagrams **36**

B Temperatures **46**

C Confusion Matrices **48**

D Nutzung von KI **55**

A Reliability Diagrams

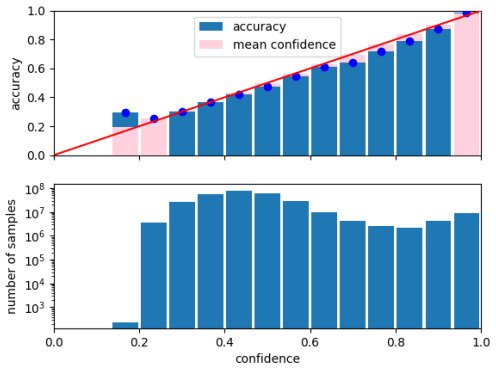


Figure A.1: Reliability diagram for the uncalibrated U-Net using SoftMax on the Data Fusion SAR dataset

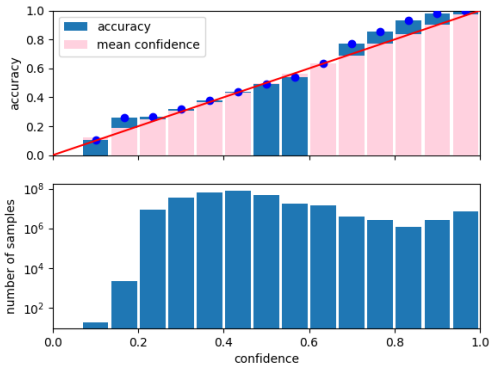


Figure A.2: Reliability diagram for the uncalibrated U-Net using Sigmoid on the Data Fusion SAR dataset

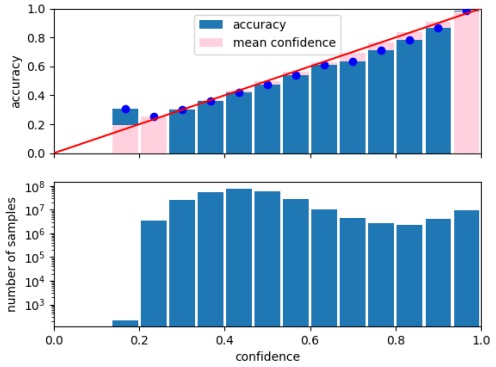


Figure A.3: Reliability diagram for the scalar temperature scaled U-Net using SoftMax on the Data Fusion SAR dataset

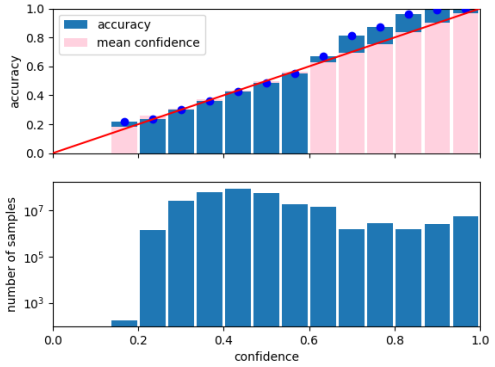


Figure A.4: Reliability diagram for the scalar temperature scaled U-Net using Sigmoid on the Data Fusion SAR dataset

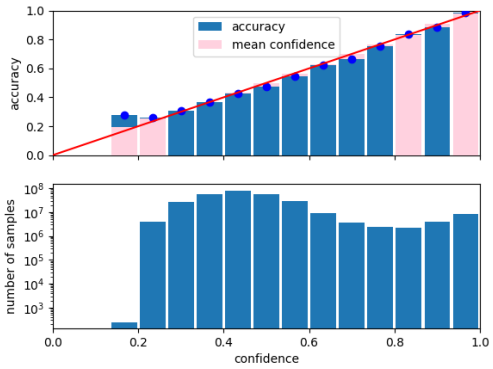


Figure A.5: Reliability diagram for the vector temperature scaled U-Net using SoftMax on the Data Fusion SAR dataset

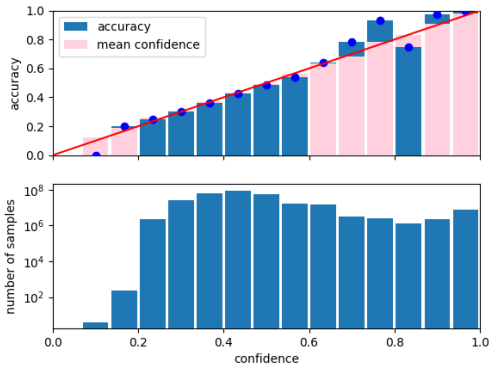


Figure A.6: Reliability diagram for the vector temperature scaled U-Net using Sigmoid on the Data Fusion SAR dataset

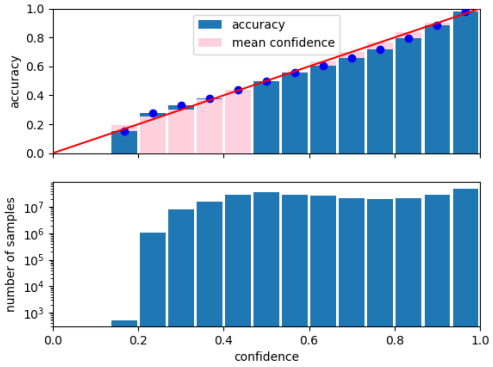


Figure A.7: Reliability diagram for the scalar temperature scaled U-Net using SoftMax on the Data Fusion RGB dataset

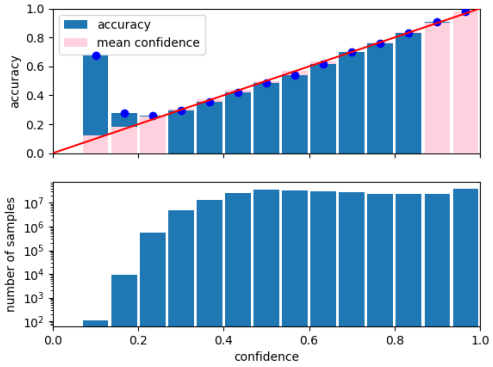


Figure A.8: Reliability diagram for the scalar temperature scaled U-Net using Sigmoid on the Data Fusion RGB dataset

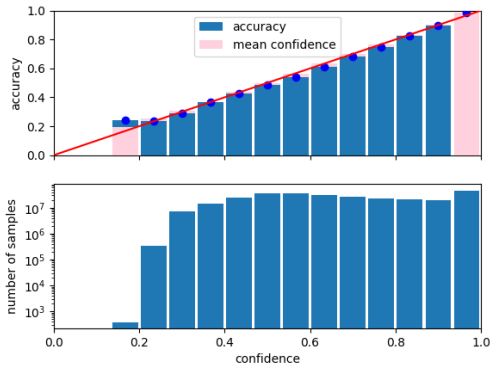


Figure A.9: Reliability diagram for the vector temperature scaled U-Net using SoftMax on the Data Fusion RGB dataset

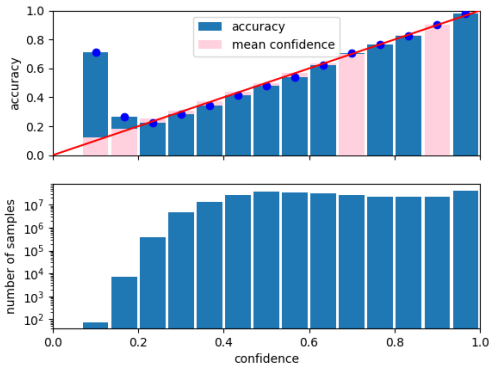


Figure A.10: Reliability diagram for the vector temperature scaled U-Net using Sigmoid on the Data Fusion RGB dataset

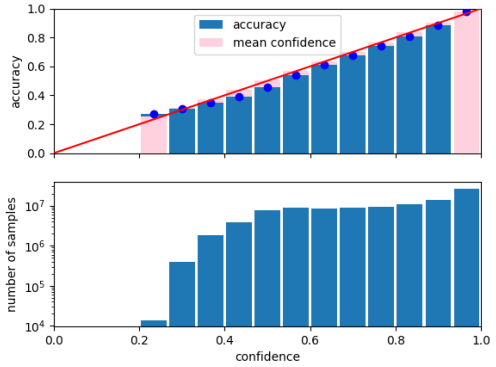


Figure A.11: Reliability diagram for the uncalibrated Deep ResUnet using SoftMax on the Potsdam dataset

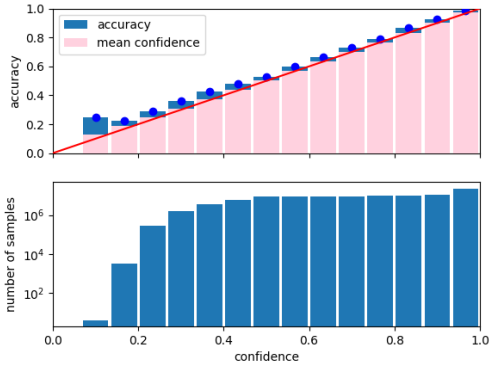


Figure A.12: Reliability diagram for the uncalibrated Deep ResUnet using Sigmoid on the Potsdam dataset

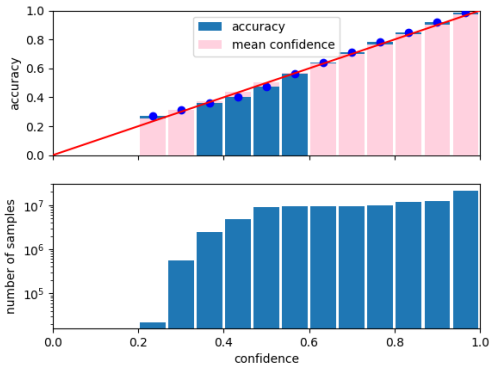


Figure A.13: Reliability diagram for the scalar temperature scaled Deep ResUnet using SoftMax on the Potsdam dataset

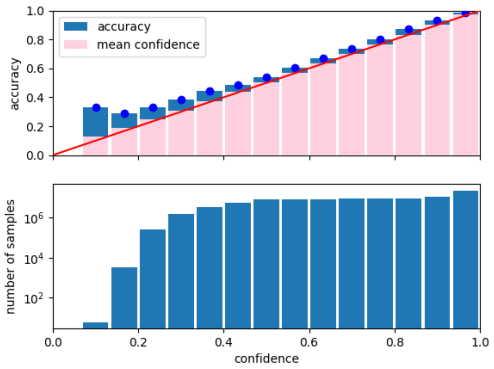


Figure A.14: Reliability diagram for the scalar temperature scaled Deep ResUnet using Sigmoid on the Potsdam dataset

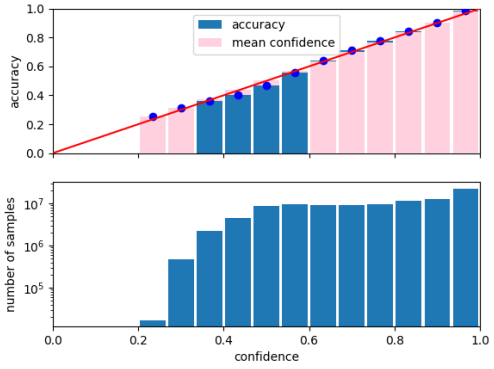


Figure A.15: Reliability diagram for the vector temperature scaled Deep ResUnet using SoftMax on the Potsdam dataset

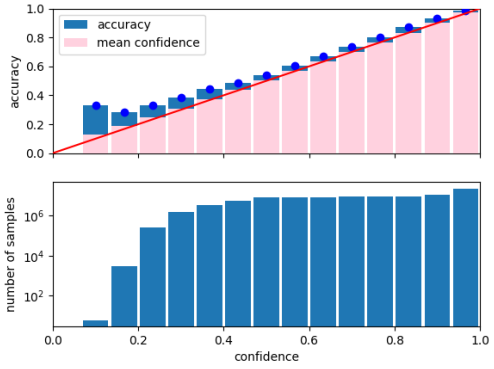


Figure A.16: Reliability diagram for the vector temperature scaled Deep ResUnet using Sigmoid on the Potsdam dataset

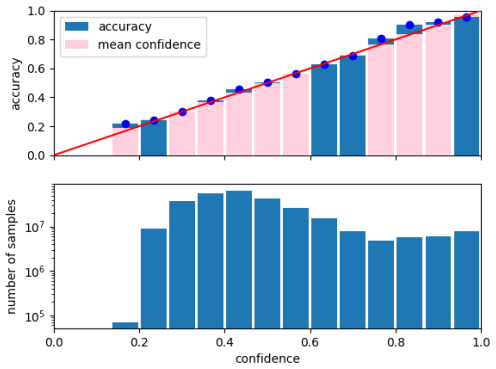


Figure A.17: Reliability diagram for the uncalibrated Deep ResUnet using SoftMax on the Data Fusion SAR dataset

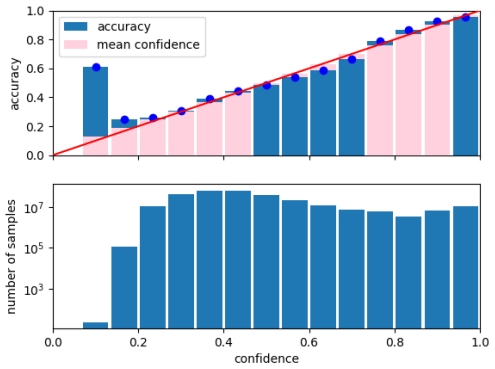


Figure A.18: Reliability diagram for the uncalibrated Deep ResUnet using Sigmoid on the Data Fusion SAR dataset

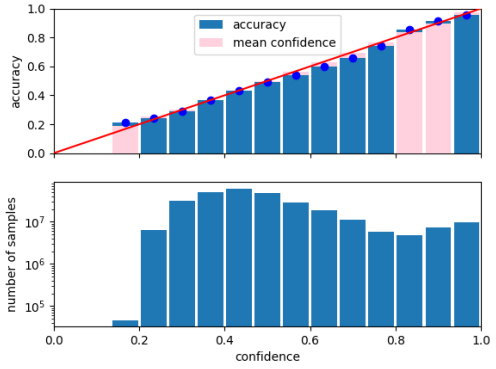


Figure A.19: Reliability diagram for the scalar temperature scaled Deep ResUnet using SoftMax on the Data Fusion SAR dataset

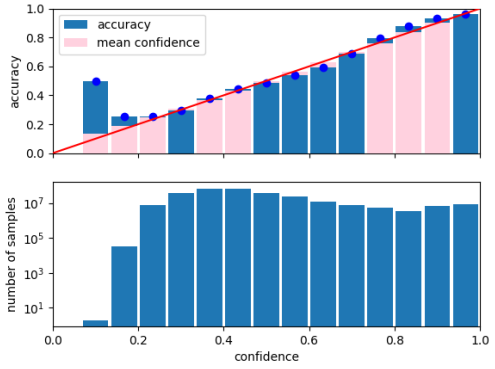


Figure A.20: Reliability diagram for the scalar temperature scaled Deep ResUnet using Sigmoid on the Data Fusion SAR dataset

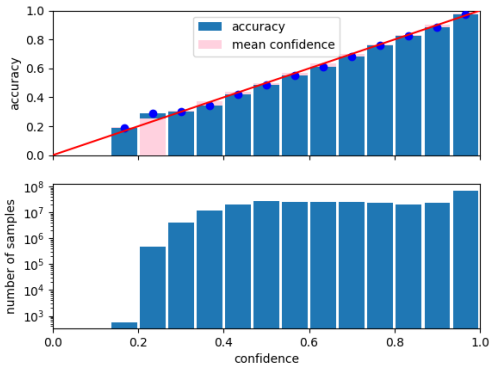


Figure A.21: Reliability diagram for the uncalibrated Deep ResUnet using SoftMax on the Data Fusion RGB dataset

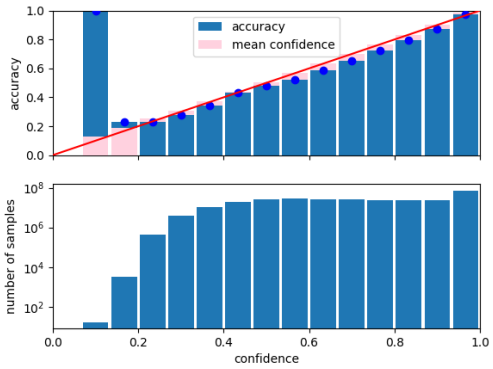


Figure A.22: Reliability diagram for the uncalibrated Deep ResUnet using Sigmoid on the Data Fusion RGB dataset

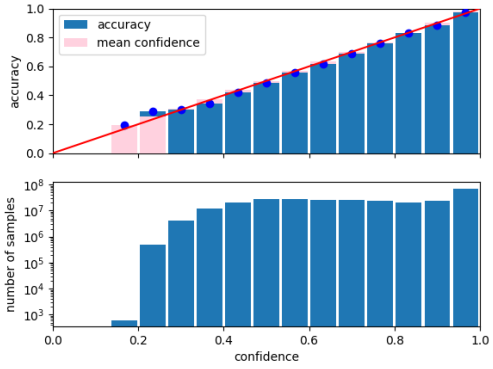


Figure A.23: Reliability diagram for the scalar temperature scaled Deep ResUnet using SoftMax on the Data Fusion RGB dataset

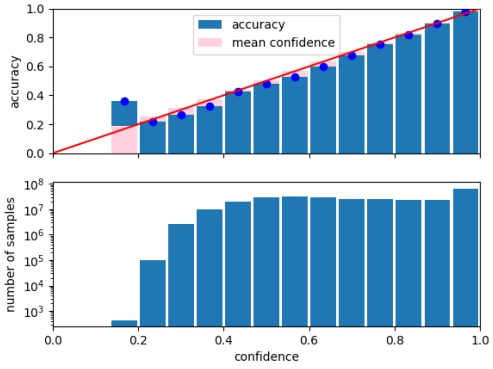


Figure A.24: Reliability diagram for the scalar temperature scaled Deep ResUnet using Sigmoid on the Data Fusion RGB dataset

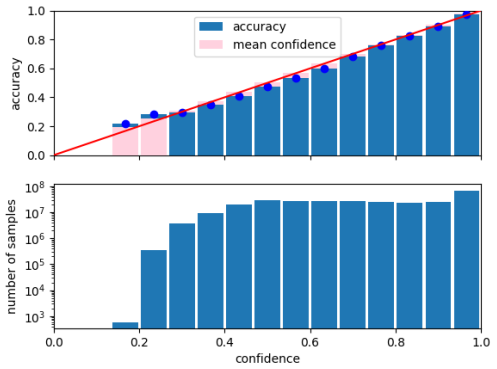


Figure A.25: Reliability diagram for the vector temperature scaled Deep ResUnet using SoftMax on the Data Fusion RGB dataset

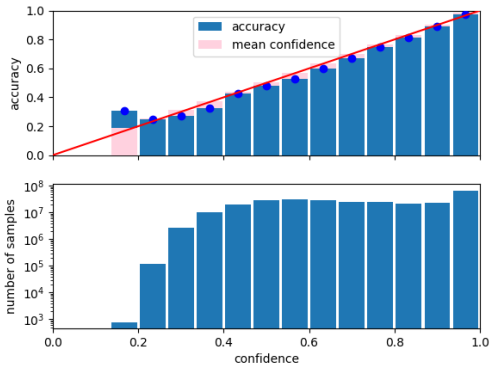


Figure A.26: Reliability diagram for the vector temperature scaled Deep ResUnet using Sigmoid on the Data Fusion RGB dataset

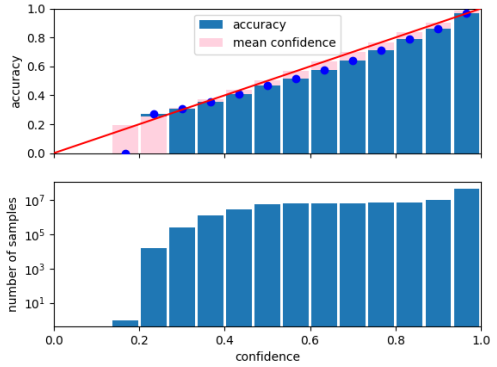


Figure A.27: Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using SoftMax on the Potsdam dataset

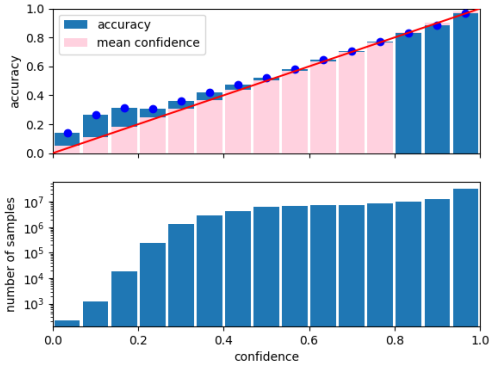


Figure A.28: Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using Sigmoid on the Potsdam dataset

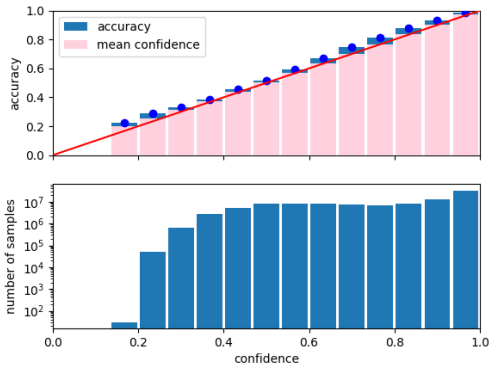


Figure A.29: Reliability diagram for the scalar temperature scaled AtrousDenseUDeconvNet using SoftMax on the Potsdam dataset

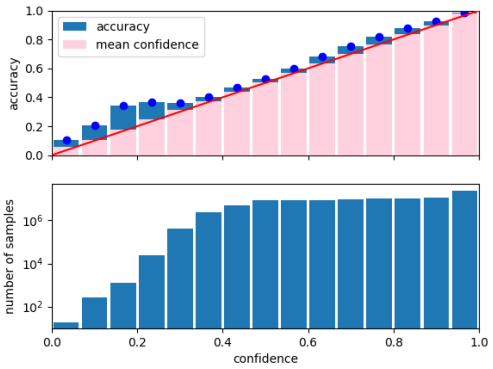


Figure A.30: Reliability diagram for the scalar temperature scaled AtrousDenseUDeconvNet using Sigmoid on the Potsdam dataset

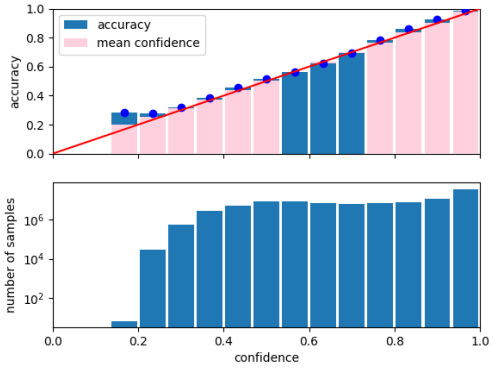


Figure A.31: Reliability diagram for the vector temperature scaled AtrousDenseUDeconvNet using SoftMax on the Potsdam dataset

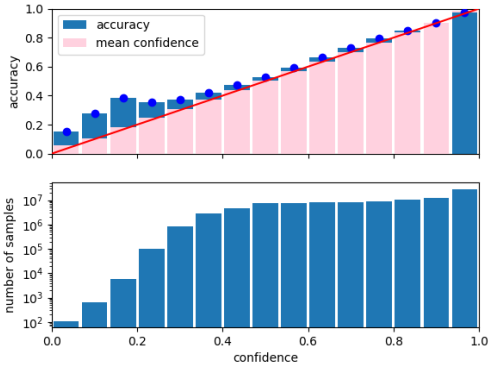


Figure A.32: Reliability diagram for the vector temperature scaled AtrousDenseUDeconvNet using Sigmoid on the Potsdam dataset

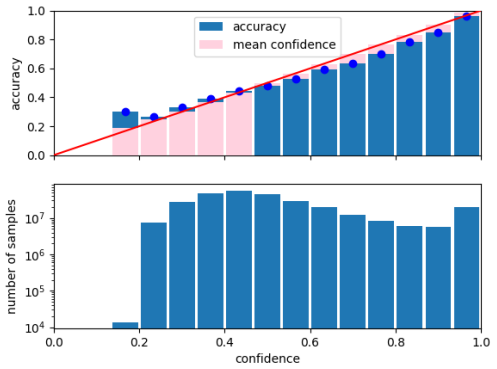


Figure A.33: Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using SoftMax on the Data Fusion SAR dataset

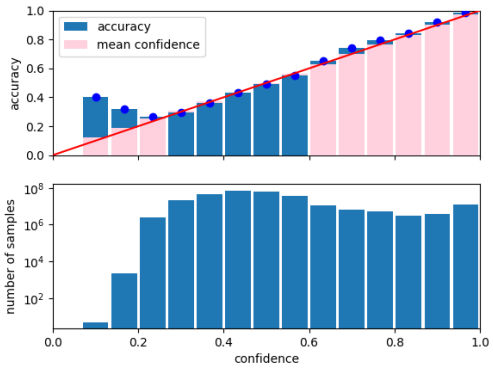


Figure A.34: Reliability diagram for the uncalibrated AtrousDenseUDeconvNet using Sigmoid on the Data Fusion SAR dataset

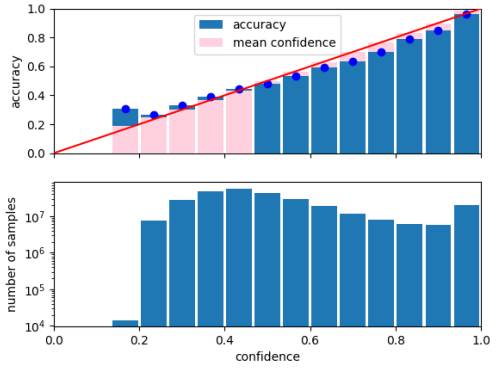


Figure A.35: Reliability diagram for the scalar temperature scaled AtrousDenseUDeconvNet using SoftMax on the Data Fusion SAR dataset

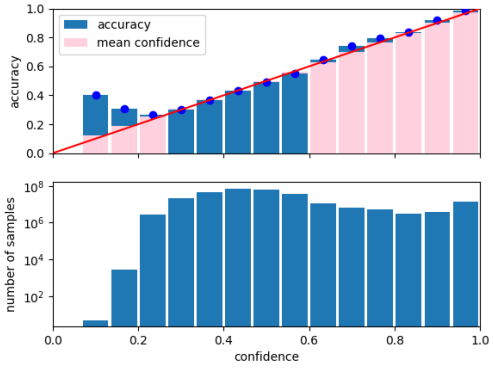


Figure A.36: Reliability diagram for the scalar temperature scaled AtrousDenseUDeconvNet using Sigmoid on the Data Fusion SAR dataset

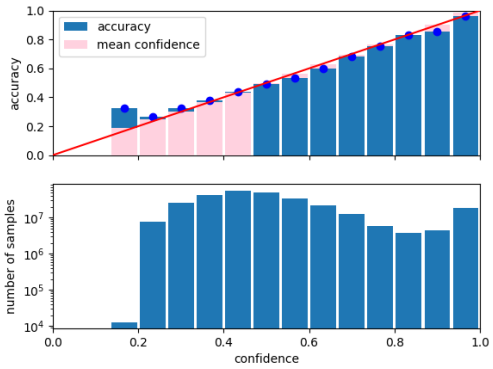


Figure A.37: Reliability diagram for the vector temperature scaled AtrousDenseUDeconvNet using SoftMax on the Data Fusion SAR dataset

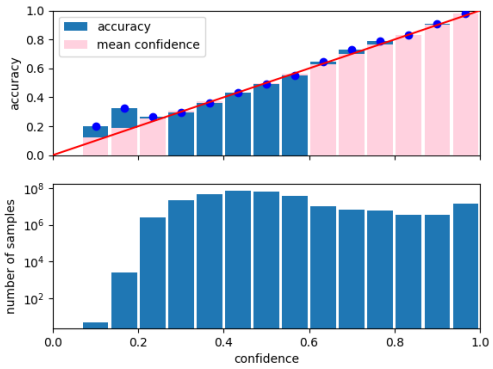


Figure A.38: Reliability diagram for the vector temperature scaled AtrousDenseUDeconvNet using Sigmoid on the Data Fusion SAR dataset

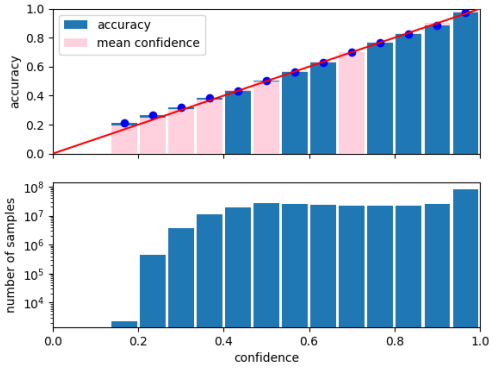


Figure A.39: Reliability diagram for the scalar temperature scaled AtrousDenseUDeconvNet using SoftMax on the Data Fusion RGB dataset

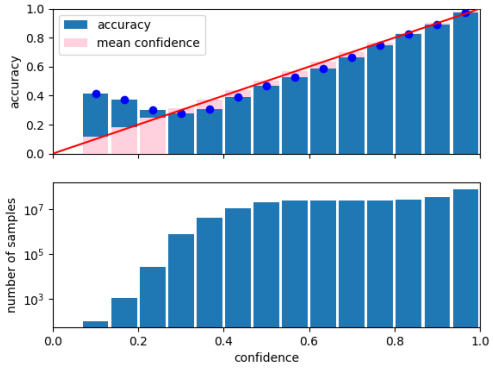


Figure A.40: Reliability diagram for the scalar temperature scaled AtrousDenseUDeconvNet using Sigmoid on the Data Fusion RGB dataset

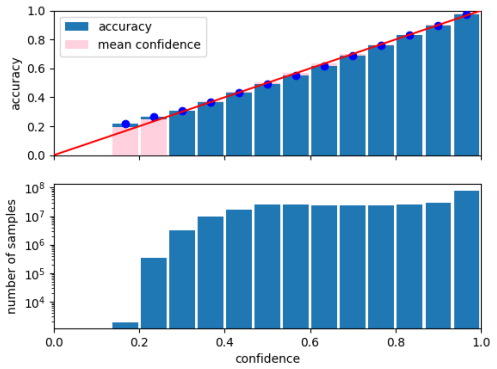


Figure A.41: Reliability diagram for the vector temperature scaled AtrousDenseUDeconvNet using SoftMax on the Data Fusion RGB dataset

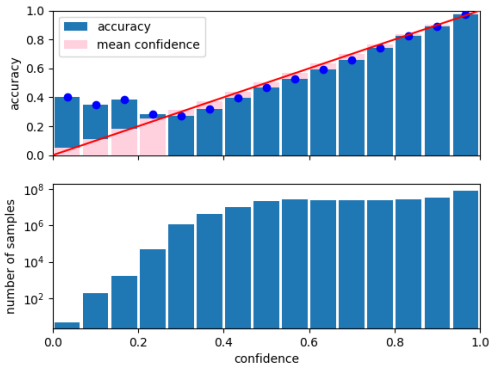


Figure A.42: Reliability diagram for the vector temperature scaled AtrousDenseUDeconvNet using Sigmoid on the Data Fusion RGB dataset

B Temperatures

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
Scalar	1.102	1.127	1.345	
Impervious Surfaces	1.012	1.014	1.029	0.349
Buildings	1.023	1.000	1.178	0.371
Low vegetation	1.207	1.075	1.329	0.142
Trees	1.019	1.078	0.880	0.116
Cars	1.001	1.001	1.001	0.023

Table B.1: Temperatures used in the calibration of the model trained on the Potsdam dataset using the SoftMax output function

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
Scalar	0.954	0.989	1.308	
Impervious Surfaces	0.998	1.001	1.011	0.349
Buildings	0.949	1.007	1.113	0.371
Low vegetation	0.964	0.988	1.117	0.142
Trees	1.014	0.979	1.139	0.116
Cars	1.000	1.000	1.000	0.023

Table B.2: Temperatures used in the calibration of the model trained on the Potsdam dataset using the Sigmoid output function

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
Scalar	0.993	0.929	1.005	
Bareland	1.000	1.000	1.000	0.015
Rangeland	1.022	1.002	1.014	0.236
Developed Space	0.994	0.881	0.931	0.228
Road	1.022	1.059	1.031	0.076
Tree	0.997	0.989	1.000	0.141
Water	1.012	0.908	1.104	0.118
Agriculture Land	0.996	0.890	1.024	0.067
Building	1.048	1.018	1.016	0.117

Table B.3: Temperatures used in the calibration of the model trained on the Data Fusion SAR dataset using the SoftMax output function

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
Scalar	1.158	1.056	0.989	
Bareland	1.000	1.000	1.000	0.015
Rangeland	1.000	0.999	1.000	0.236
Developed Space	1.208	1.084	1.026	0.228
Road	1.178	1.053	1.006	0.076
Tree	1.000	0.993	1.000	0.141
Water	1.052	1.024	0.897	0.118
Agriculture Land	0.924	0.982	0.938	0.067
Building	1.016	1.012	0.953	0.117

Table B.4: Temperatures used in the calibration of the model trained on the Data Fusion SAR dataset using the Sigmoid output function

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
Scalar	1.040	1.009	1.056	
Bareland	1.000	1.000	1.000	0.017
Rangeland	1.070	0.992	1.019	0.238
Developed Space	1.189	1.013	1.055	0.228
Road	0.949	0.953	0.963	0.074
Tree	0.940	0.989	1.026	0.142
Water	1.070	1.062	1.154	0.116
Agriculture Land	0.950	1.043	1.041	0.070
Building	0.995	1.038	1.034	0.114

Table B.5: Temperatures used in the calibration of the model trained on the Data Fusion RGB dataset using the SoftMax output function

	U-Net	Deep ResUnet	AtrousDenseUDeconvNet	Sample %
Scalar	1.027	1.111	1.288	
Bareland	1.000	1.000	1.000	0.017
Rangeland	1.000	1.001	1.001	0.238
Developed Space	1.012	1.114	1.397	0.228
Road	1.144	1.129	1.373	0.074
Tree	1.002	1.011	1.099	0.142
Water	0.969	1.077	1.201	0.116
Agriculture Land	0.991	1.033	1.059	0.070
Building	1.006	1.028	1.069	0.114

Table B.6: Temperatures used in the calibration of the model trained on the Data Fusion RGB dataset using the Sigmoid output function

C Confusion Matrices

	IS	BU	LV	TR	CA	Total
Impervious Surfaces	0.831	0.028	0.128	0.010	0.004	35051298
Buildings	0.165	0.725	0.095	0.009	0.004	37275908
Low Vegetation	0.081	0.013	0.832	0.072	0.001	14264583
Trees	0.211	0.010	0.515	0.254	0.010	11693204
Cars	0.127	0.099	0.047	0.010	0.716	2279656

Table C.1: Confusion Matrix on the Potsdam dataset for U-Net using SoftMax

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.000	0.685	0.054	0.000	0.141	0.018	0.092	0.010	4346918
Rangeland	0.000	0.497	0.187	0.000	0.211	0.008	0.056	0.041	67176971
Developed Space	0.000	0.230	0.485	0.000	0.048	0.009	0.026	0.202	64849707
Road	0.000	0.221	0.489	0.000	0.070	0.021	0.009	0.190	21580152
Tree	0.000	0.227	0.133	0.000	0.559	0.004	0.002	0.074	40050002
Water	0.000	0.191	0.142	0.000	0.020	0.492	0.133	0.022	33639478
Agriculture Land	0.000	0.399	0.024	0.000	0.043	0.020	0.511	0.003	18894984
Building	0.000	0.036	0.421	0.000	0.020	0.003	0.001	0.518	33350537

Table C.2: Confusion Matrix on the DF SAR dataset for U-Net using SoftMax

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.163	0.399	0.254	0.007	0.002	0.006	0.134	0.035	4735464
Rangeland	0.006	0.749	0.068	0.002	0.101	0.005	0.063	0.005	67693539
Developed Space	0.001	0.149	0.592	0.062	0.010	0.006	0.015	0.165	64775329
Road	0.000	0.074	0.454	0.284	0.016	0.012	0.009	0.151	21117598
Tree	0.000	0.171	0.015	0.001	0.800	0.005	0.004	0.004	40424118
Water	0.000	0.039	0.063	0.010	0.008	0.835	0.026	0.020	33024005
Agriculture Land	0.085	0.217	0.130	0.003	0.011	0.038	0.517	0.000	19757911
Building	0.000	0.027	0.209	0.039	0.008	0.004	0.001	0.712	32510546

Table C.3: Confusion Matrix on the DF RGB dataset for U-Net using SoftMax

	IS	BU	LV	TR	CA	Total
Impervious Surfaces	0.789	0.104	0.075	0.029	0.004	35051298
Buildings	0.093	0.866	0.035	0.004	0.002	37275908
Low Vegetation	0.118	0.026	0.642	0.214	0.000	14264583
Trees	0.272	0.022	0.286	0.410	0.010	11693204
Cars	0.119	0.159	0.024	0.000	0.670	2279656

Table C.4: Confusion Matrix on the Potsdam dataset for U-Net using Sigmoid

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.000	0.694	0.061	0.000	0.051	0.021	0.164	0.009	4346918
Rangeland	0.000	0.489	0.168	0.000	0.190	0.012	0.091	0.051	67176971
Developed Space	0.000	0.218	0.448	0.000	0.057	0.010	0.031	0.235	64849707
Road	0.000	0.218	0.449	0.000	0.091	0.013	0.007	0.222	21580152
Tree	0.000	0.295	0.098	0.000	0.528	0.004	0.004	0.072	40050002
Water	0.000	0.152	0.146	0.000	0.029	0.539	0.108	0.026	33639478
Agriculture Land	0.000	0.380	0.030	0.000	0.035	0.035	0.518	0.003	18894984
Building	0.000	0.044	0.326	0.000	0.042	0.002	0.001	0.585	33350537

Table C.5: Confusion Matrix on the DF SAR dataset for U-Net using Sigmoid

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.002	0.371	0.330	0.002	0.000	0.004	0.268	0.024	4735464
Rangeland	0.000	0.784	0.096	0.002	0.063	0.002	0.051	0.002	67693539
Developed Space	0.000	0.113	0.770	0.031	0.005	0.004	0.019	0.058	64775329
Road	0.000	0.080	0.674	0.178	0.011	0.004	0.005	0.048	21117598
Tree	0.000	0.273	0.028	0.001	0.693	0.003	0.001	0.002	40424118
Water	0.000	0.051	0.085	0.012	0.006	0.815	0.021	0.010	33024005
Agriculture Land	0.001	0.280	0.128	0.001	0.006	0.017	0.565	0.000	19757911
Building	0.000	0.025	0.408	0.042	0.005	0.003	0.000	0.516	32510546

Table C.6: Confusion Matrix on the DF RGB dataset for U-Net using Sigmoid

	IS	BU	LV	TR	CA	Total
Impervious Surfaces	0.767	0.041	0.135	0.053	0.004	35051298
Buildings	0.113	0.742	0.108	0.026	0.011	37275908
Low Vegetation	0.035	0.006	0.859	0.098	0.001	14264583
Trees	0.080	0.012	0.309	0.590	0.009	11693204
Cars	0.084	0.024	0.042	0.020	0.830	2279656

Table C.7: Confusion Matrix on the Potsdam dataset for Deep ResUnet using SoftMax

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.000	0.618	0.106	0.000	0.069	0.015	0.181	0.011	4346918
Rangeland	0.000	0.453	0.219	0.001	0.185	0.016	0.089	0.036	67176971
Developed Space	0.000	0.194	0.522	0.002	0.055	0.021	0.029	0.176	64849707
Road	0.000	0.192	0.471	0.006	0.091	0.043	0.020	0.176	21580152
Tree	0.000	0.240	0.116	0.001	0.554	0.010	0.020	0.059	40050002
Water	0.000	0.120	0.122	0.001	0.037	0.684	0.016	0.021	33639478
Agriculture Land	0.000	0.252	0.024	0.000	0.053	0.028	0.639	0.002	18894984
Building	0.000	0.057	0.410	0.002	0.042	0.010	0.003	0.476	33350537

Table C.8: Confusion Matrix on the DF SAR dataset for Deep ResUnet using SoftMax

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.046	0.199	0.478	0.006	0.005	0.013	0.228	0.025	4735464
Rangeland	0.001	0.721	0.123	0.008	0.092	0.012	0.041	0.002	67693539
Developed Space	0.001	0.076	0.731	0.098	0.011	0.012	0.015	0.056	64775329
Road	0.000	0.038	0.331	0.570	0.013	0.009	0.006	0.032	21117598
Tree	0.000	0.114	0.012	0.001	0.864	0.007	0.002	0.001	40424118
Water	0.000	0.036	0.068	0.008	0.009	0.854	0.012	0.012	33024005
Agriculture Land	0.045	0.196	0.157	0.005	0.017	0.050	0.531	0.000	19757911
Building	0.001	0.019	0.292	0.065	0.009	0.007	0.001	0.607	32510546

Table C.9: Confusion Matrix on the DF RGB dataset for Deep ResUnet using SoftMax

	IS	BU	LV	TR	CA	Total
Impervious Surfaces	0.737	0.141	0.084	0.022	0.006	35051298
Buildings	0.058	0.847	0.059	0.011	0.006	37275908
Low Vegetation	0.080	0.020	0.807	0.088	0.002	14264583
Trees	0.139	0.028	0.304	0.514	0.010	11693204
Cars	0.075	0.086	0.014	0.001	0.812	2279656

Table C.10: Confusion Matrix on the Potsdam dataset for Deep ResUnet using Sigmoid

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.012	0.634	0.092	0.000	0.130	0.015	0.110	0.007	4346918
Rangeland	0.003	0.610	0.154	0.015	0.124	0.016	0.056	0.021	67176971
Developed Space	0.002	0.300	0.476	0.026	0.043	0.024	0.017	0.113	64849707
Road	0.001	0.329	0.373	0.065	0.061	0.040	0.015	0.116	21580152
Tree	0.003	0.468	0.075	0.014	0.385	0.013	0.010	0.032	40050002
Water	0.000	0.176	0.093	0.007	0.025	0.674	0.013	0.012	33639478
Agriculture Land	0.019	0.306	0.017	0.002	0.040	0.030	0.584	0.002	18894984
Building	0.000	0.122	0.458	0.021	0.047	0.013	0.001	0.338	33350537

Table C.11: Confusion Matrix on the DF SAR dataset for Deep ResUnet using Sigmoid

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.045	0.190	0.520	0.001	0.005	0.015	0.199	0.025	4735464
Rangeland	0.001	0.677	0.170	0.001	0.082	0.015	0.048	0.006	67693539
Developed Space	0.001	0.048	0.821	0.019	0.007	0.007	0.015	0.082	64775329
Road	0.000	0.029	0.590	0.259	0.008	0.012	0.009	0.093	21117598
Tree	0.000	0.108	0.025	0.000	0.848	0.012	0.004	0.004	40424118
Water	0.000	0.026	0.090	0.001	0.005	0.854	0.014	0.009	33024005
Agriculture Land	0.047	0.142	0.176	0.003	0.011	0.065	0.555	0.001	19757911
Building	0.000	0.009	0.305	0.011	0.006	0.005	0.001	0.663	32510546

Table C.12: Confusion Matrix on the DF RGB dataset for Deep ResUnet using Sigmoid

	IS	BU	LV	TR	CA	Total
Impervious Surfaces	0.870	0.047	0.072	0.006	0.004	35051298
Buildings	0.090	0.859	0.043	0.003	0.005	37275908
Low Vegetation	0.121	0.024	0.813	0.041	0.001	14264583
Trees	0.232	0.020	0.382	0.354	0.012	11693204
Cars	0.151	0.023	0.008	0.001	0.817	2279656

Table C.13: Confusion Matrix on the Potsdam dataset for AtrousDenseUDeconvNet using SoftMax

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.000	0.669	0.086	0.000	0.123	0.030	0.085	0.007	4346918
Rangeland	0.000	0.516	0.219	0.003	0.206	0.009	0.020	0.028	67176971
Developed Space	0.000	0.182	0.595	0.003	0.045	0.015	0.005	0.155	64849707
Road	0.000	0.156	0.584	0.008	0.074	0.018	0.003	0.157	21580152
Tree	0.000	0.177	0.121	0.004	0.648	0.006	0.001	0.044	40050002
Water	0.000	0.126	0.170	0.001	0.015	0.672	0.005	0.012	33639478
Agriculture Land	0.000	0.446	0.023	0.000	0.044	0.048	0.437	0.002	18894984
Building	0.000	0.025	0.482	0.002	0.041	0.005	0.000	0.445	33350537

Table C.14: Confusion Matrix on the DF SAR dataset for AtrousDenseUDeconvNet using SoftMax

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.139	0.330	0.270	0.017	0.015	0.024	0.187	0.017	4735464
Rangeland	0.007	0.764	0.078	0.011	0.102	0.006	0.026	0.006	67693539
Developed Space	0.006	0.078	0.685	0.099	0.009	0.015	0.006	0.102	64775329
Road	0.001	0.030	0.276	0.610	0.013	0.006	0.003	0.062	21117598
Tree	0.000	0.099	0.013	0.003	0.875	0.002	0.004	0.003	40424118
Water	0.003	0.035	0.045	0.008	0.014	0.880	0.011	0.004	33024005
Agriculture Land	0.068	0.207	0.085	0.005	0.019	0.037	0.580	0.000	19757911
Building	0.000	0.009	0.205	0.027	0.006	0.012	0.000	0.741	32510546

Table C.15: Confusion Matrix on the DF RGB dataset for AtrousDenseUDeconvNet using SoftMax

	IS	BU	LV	TR	CA	Total
Impervious Surfaces	0.692	0.199	0.056	0.048	0.004	35051298
Buildings	0.022	0.939	0.021	0.014	0.001	37275908
Low Vegetation	0.077	0.057	0.665	0.199	0.000	14264583
Trees	0.118	0.031	0.132	0.708	0.006	11693204
Cars	0.087	0.110	0.019	0.009	0.741	2279656

Table C.16: Confusion Matrix on the Potsdam dataset for AtrousDenseUDeconvNet using Sigmoid

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.000	0.740	0.058	0.000	0.074	0.037	0.087	0.004	4346918
Rangeland	0.000	0.598	0.196	0.000	0.130	0.017	0.042	0.017	67176971
Developed Space	0.000	0.244	0.594	0.000	0.023	0.018	0.011	0.109	64849707
Road	0.000	0.247	0.566	0.000	0.043	0.018	0.004	0.122	21580152
Tree	0.000	0.336	0.138	0.000	0.486	0.004	0.002	0.033	40050002
Water	0.000	0.158	0.149	0.000	0.017	0.646	0.024	0.007	33639478
Agriculture Land	0.000	0.407	0.017	0.000	0.018	0.035	0.522	0.001	18894984
Building	0.000	0.052	0.585	0.000	0.020	0.004	0.000	0.340	33350537

Table C.17: Confusion Matrix on the DF SAR dataset for AtrousDenseUDeconvNet using Sigmoid

	BL	RL	DS	RD	TR	WA	AL	BU	Total
Bareland	0.065	0.383	0.279	0.026	0.006	0.012	0.211	0.017	4735464
Rangeland	0.002	0.792	0.083	0.007	0.074	0.003	0.033	0.004	67693539
Developed Space	0.004	0.073	0.746	0.060	0.006	0.006	0.015	0.091	64775329
Road	0.000	0.036	0.369	0.515	0.008	0.005	0.005	0.062	21117598
Tree	0.000	0.118	0.014	0.002	0.855	0.001	0.006	0.003	40424118
Water	0.004	0.041	0.045	0.005	0.005	0.864	0.024	0.012	33024005
Agriculture Land	0.070	0.195	0.068	0.005	0.011	0.022	0.630	0.000	19757911
Building	0.000	0.012	0.193	0.013	0.004	0.003	0.001	0.773	32510546

Table C.18: Confusion Matrix on the DF RGB dataset for AtrousDenseUDeconvNet using Sigmoid

D Nutzung von KI

OpenAI, Chat GPT - GPT 5, sprachliche Überprüfung und Verbesserung der Texte
Google, Gemini 3 Pro, sprachliche Überprüfung und Verbesserung der Texte
Google, Gemini 3 Fast, sprachliche Überprüfung und Verbesserung der Texte
DeepL, DeepL Write, sprachliche Überprüfung und Verbesserung der Texte